Universidad de Las Palmas de Gran Canaria

Grado en Ciencia e Ingeniería de Datos

Big Data

Academic year 2026–2027

Boogle Engine

Stage 2 — Service-Oriented Architecture

Final report

Group bigdata-devs

Full name	Student ID	GitHub account
Alejandro Delgado Valera	46261876K	`aledelgadoo`
Aythami Lorenzo Padilla	45618618G	`aythamilorenzo`
Carlos Montesdeoca Vega	45368153D	`CarlosMontesdeoca21`
Javier Ruano Hernández	78598634K	`javierruanohdez`
Tomás Santana Suárez	45376459N	`TemiArtemi`

GitHub repository: https://github.com/bigdata-devs/stage_2

Report date: 3 November 2026

Code described: branch `develop` at `790f75d`. Official benchmark session: commit `ade7245`, 1 November 2026.

Contents

2 Introduction and Objectives **1**

3 System Architecture **1**
3.1 Services and responsibilities . . . 1
3.2 Coordination mechanism . . . 3
3.3 Processes, replicas and threads . . . 3
3.4 Shared storage . . . 4
3.5 Single-host deployment . . . 5

4 API Design and Contracts **5**
4.1 Consumers and use cases . . . 5
4.2 Endpoint inventory . . . 5
4.3 Front-end contract . . . 5
4.4 Internal contracts . . . 7
4.5 Error model and status codes . . . 7
4.6 Non-HTTP contracts: shared state . . . 7
4.7 Exposure and cross-origin access . . . 8
4.8 Specification and validation evidence . . . 8

5 Microservice Implementation **9**
5.1 Technology stack and code organisation . . . 9
5.2 Ingestion Service . . . 10
5.3 Indexing Service . . . 10
5.4 Search Service . . . 10
5.5 Control Module . . . 11
5.6 Readiness, task ownership, retries, idempotency and publication . . . 11
5.7 Testing . . . 11

6 Design Decisions **12**
6.1 Coordination mechanism . . . 12
6.2 Single-machine deployment . . . 12
6.3 API design . . . 13
6.4 Storage selection . . . 13
6.5 Concurrency model and protection of shared state . . . 13
6.6 Replica counts, thread settings and ports . . . 14
6.7 Where replication and threading help, and where they add contention . . . 14

7 Benchmarks and Results **15**
7.1 Experimental setup . . . 15
7.2 Indexing throughput . . . 16
7.3 Query latency and concurrency . . . 18
7.4 CPU and memory utilisation per process and host . . . 19
7.5 Effects of replication . . . 21
7.6 Effects of threading . . . 21
7.7 Coordination overhead . . . 23
7.8 Bottlenecks and performance limits of the single host . . . 23

7.9 Threats to validity 23

8 Conclusions and Future Improvements **24**

8.1 Strengths 24

8.2 Weaknesses 24

8.3 From the measurements to the Stage 3 evolution 25

A Evidence Provenance **26**

2 Introduction and Objectives

Stage 1 built the data layer of the search engine and measured it: four behaviourally identical implementations of the same pipeline were compared on one host, and the group selected **Go**, the **batch-based datalake** and the **MongoDB inverted index** for Stage 2. The selection of MongoDB was explicitly provisional. It did not rest on the Stage 1 measurements, where file-based indexes answered queries faster, but on a requirement those measurements could not cover: several independent processes reading and updating one shared index. The Stage 1 report therefore left Stage 2 a concrete obligation, namely to measure MongoDB under concurrent access before confirming the choice.

Stage 2 turns that data layer into a *service-oriented architecture* (SOA). The monolithic pipeline is decomposed into three autonomous processes (the Ingestion, Indexing and Search services) that cooperate through REST interfaces with JSON payloads, an explicit coordination mechanism decides which work is ready and when its result may be published, and every process (replicas, the reverse proxy, the database server and the coordinator) runs on a single host. The objectives of this stage, and where this report shows that each was met, are the following:

O1 Decomposition. Separate ingestion, indexing and search into independently deployable services with their own REST APIs and internal data model (Sections 3 and 5).

O2 Explicit coordination. Guarantee that a book is indexed only after its ingestion has completed, that search reads only published data, and that repeated or concurrent requests never duplicate or corrupt records. The design combines classical orchestration by a Control Module with persistent completion markers, a publication state in the datamart and a cross-replica lease (Sections 3.2 and 6.1).

O3 Contracts. Distinguish the front-end-facing API from the internal and operational interfaces, publish versioned OpenAPI specifications and validate the running services against them (Section 4).

O4 Concurrency. Serve many concurrent users and account for the three levels of concurrency the assignment separates: client concurrency, process replicas and threads within a process (Sections 3.3 and 6.5).

O5 Measurement. Quantify indexing throughput, query latency, CPU and memory per process and for the host, the effect of replication and threading, and the coordination overhead, and locate the limits of the single-machine deployment (Section 7).

O6 Preparation for Stage 3. Identify every assumption that only holds on one machine and describe how the architecture will evolve into a cluster (Section 8).

Key results

- **Contracts hold:** 19/19 API checks, 365 Go tests and both end-to-end scripts pass.
- **MongoDB saturates first:** it uses 94–97% of query CPU, and search plateaus at about 250 req/s.
- **Replication does not help on one host:** two Search replicas change throughput by under 5%.
- **Indexing is contention-bound:** 48% parallel efficiency at ten concurrent updates.

Scope of the claims. Every performance statement in this report refers to the host described in Table 12 (one laptop-class machine running Docker under WSL2), to the datasets of Table 13 (a datamart of 300 published books for search, a corpus of 20 books for indexing) and to one run per configuration. The numbers characterise this implementation on this machine. Following the assignment, a single-machine experiment is *not* presented as evidence of cluster scalability; that question belongs to Stage 3.

3 System Architecture

3.1 Services and responsibilities

The system is composed of the three core services required by the assignment, a Control Module that coordinates them, an nginx reverse proxy that publishes the Search API, and a MongoDB server that holds the datamart. Table 1 lists every process of the deployment and Figure 1 shows how they interact; the internal structure of each service is described in Section 5.

Table 1: Processes of the Stage 2 deployment. All of them run on the single host of Table 12; "port" is the port published on that host.

Process	Responsibility	Port	Replicas	Storage access
Ingestion Service	Downloads a Project Gutenberg book, splits licence header and body, stores both in the datalake	7001	1	datalake (write)
Indexing Service	Extracts metadata and terms, writes the metadata catalogue and the inverted index, rebuilds the datamart	7002	1	datalake (read), MongoDB (write)
Search Service	Front-end API: keyword search with filters and pagination, book details, filter values	—	1 or 2	MongoDB (read)
nginx	Front door of the Search API; balances the Search replicas	7003	1	—
Control Module	Orchestrates ingestion before indexing, records progress, retries failures (command-line tool, run on demand)	—	1	control state (append)
MongoDB 7	Datamart server: `books`, `inverted_index`, `locks`	27017	1	data volume

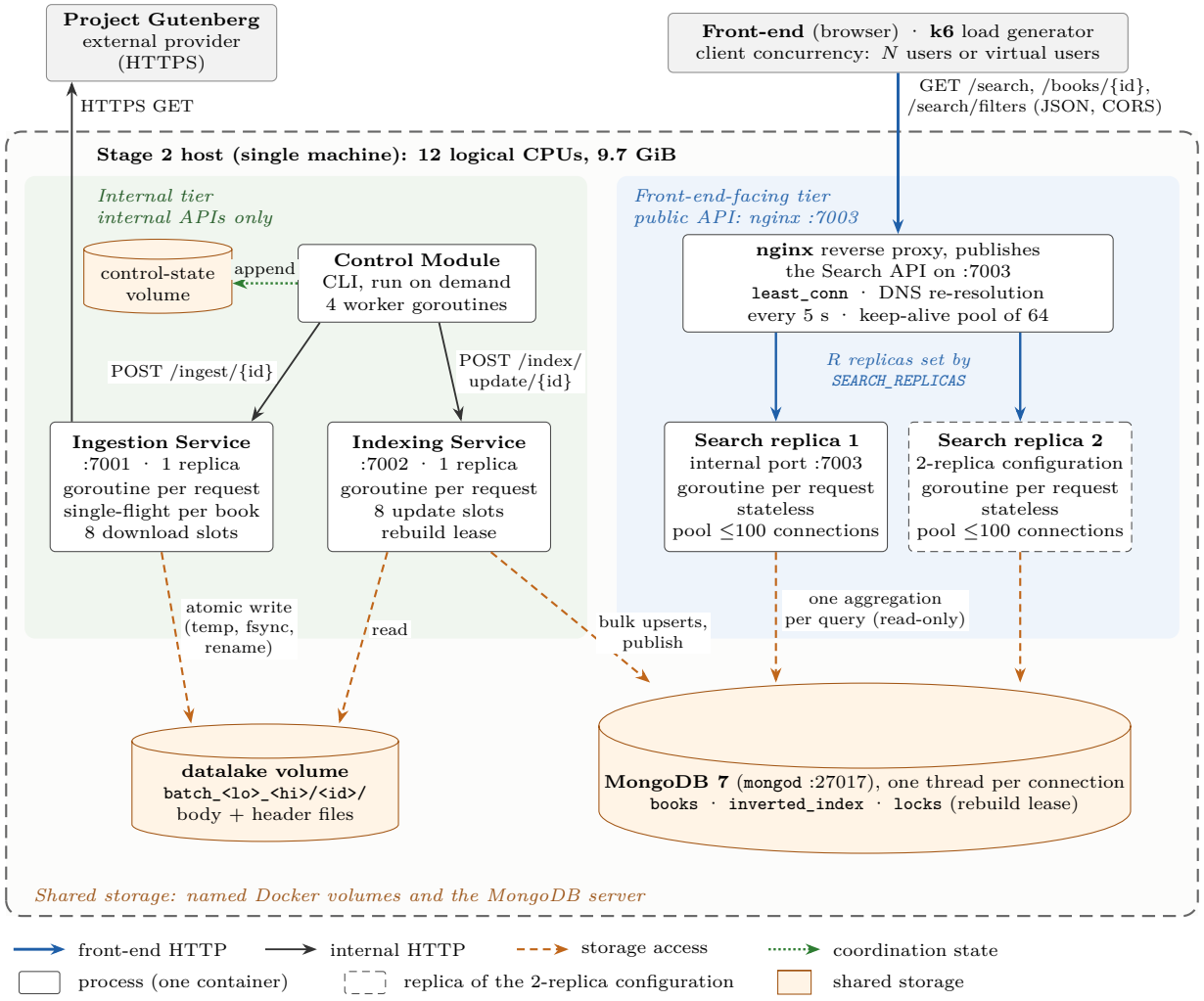


Figure 1: Deployment and interaction view of the Stage 2 architecture. The front-end API boundary separates the front-end-facing tier (nginx and the Search replicas, the only components a front-end calls) from the internal tier, whose APIs are consumed by the Control Module. Process replicas are drawn as separate boxes, and the thread model of each process is written inside its box. Ingestion and Indexing share the datalake volume, and Indexing and Search share the MongoDB datamart. During the benchmarks of Section 7 the k6 load generator also ran on this host.

3.2 Coordination mechanism

The architecture uses **classical orchestration** (the design of Figure 1 of the assignment), reinforced by three shared-state mechanisms that make the workflow safe across process restarts and replicas:

1. **Orchestration.** The Control Module hands each book to one worker, which calls the Ingestion Service and, only after a `2xx` answer, the Indexing Service; both calls are synchronous, so `2xx` means the step has *completed*.
2. **Persistent completion markers.** A book exists in the datalake only when both of its files do, and the Control Module records every completed or abandoned step in append-only control-state files.
3. **Publication state in the datamart.** A book's metadata record stays `pending` until its postings are written and then becomes `published`; search returns published books only.
4. **Cross-replica lease.** A lease document in MongoDB allows one full rebuild at a time across every Indexing replica.

The rules these mechanisms impose on shared storage are specified as non-HTTP contracts in Table 6. Figure 2 traces one book through the workflow. The dependency required by the assignment (a book becomes eligible for indexing only after its ingestion has completed) is enforced twice: by the orchestrator, which calls `POST /index/update/{id}` only after ingestion answered `2xx`, and by the Indexing Service, which refuses a book that is not complete in the datalake. Search reads a consistent published state because publication is the last write of every update and a rebuild is swapped in atomically (Section 5.3). If a process stops between completing a step and recording it, the book stays unrecorded and the next run replays the call; because every operation is idempotent, the replay converges without duplicates (Section 5.6).

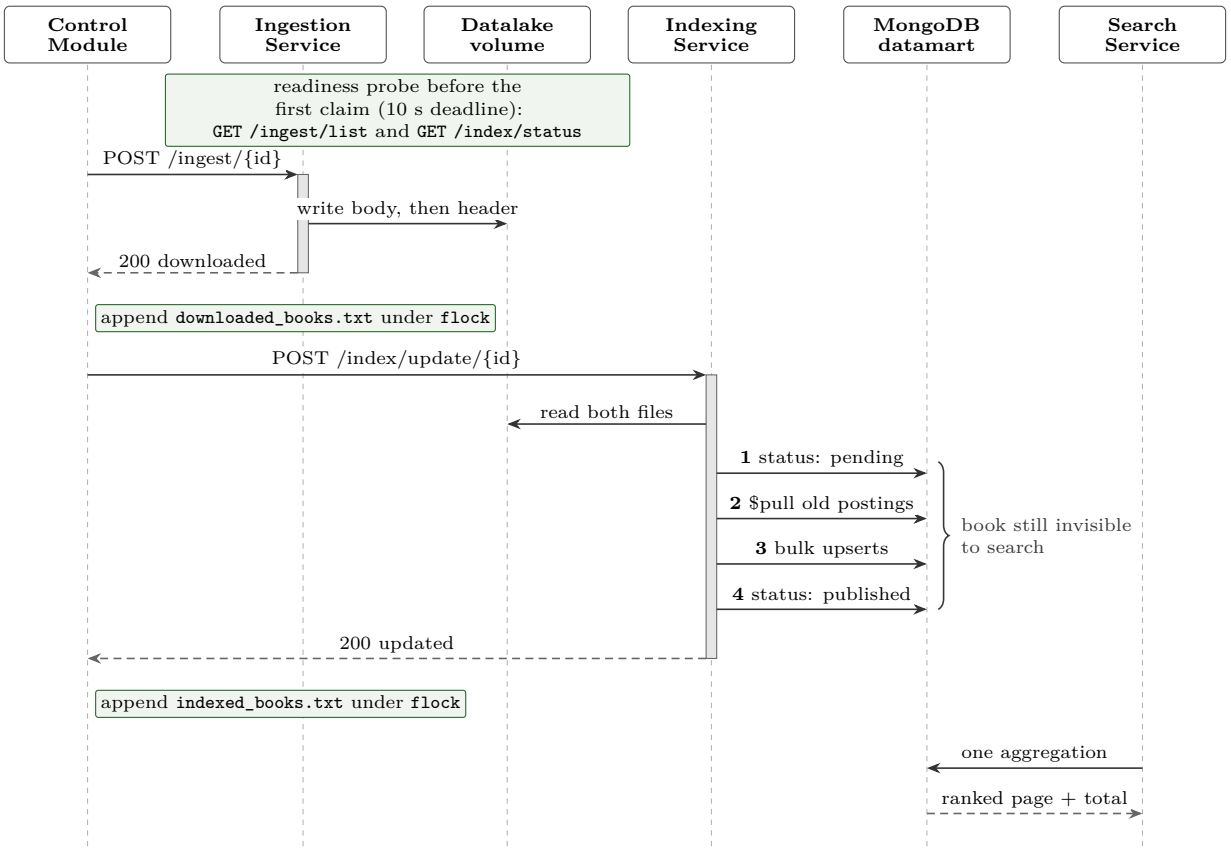


Figure 2: Lifecycle of one book under the orchestration workflow: solid arrows are requests or writes, dashed arrows responses, and green notes the readiness probe and the completion markers of the control state. Steps 1–4 are the ordered datamart writes of one index update (Section 5.3); the book becomes visible to search only at step 4, so a crash at any earlier step leaves it invisible rather than half-indexed.

3.3 Processes, replicas and threads

The assignment distinguishes three levels of concurrency, which this report keeps separate throughout: *client concurrency* (how many requests the front-end or the load generator keeps in flight), *process replicas* (how many

instances of a service run) and *threads* (how many concurrent activities one process executes). Figure 3 maps them onto the two request paths of the system.

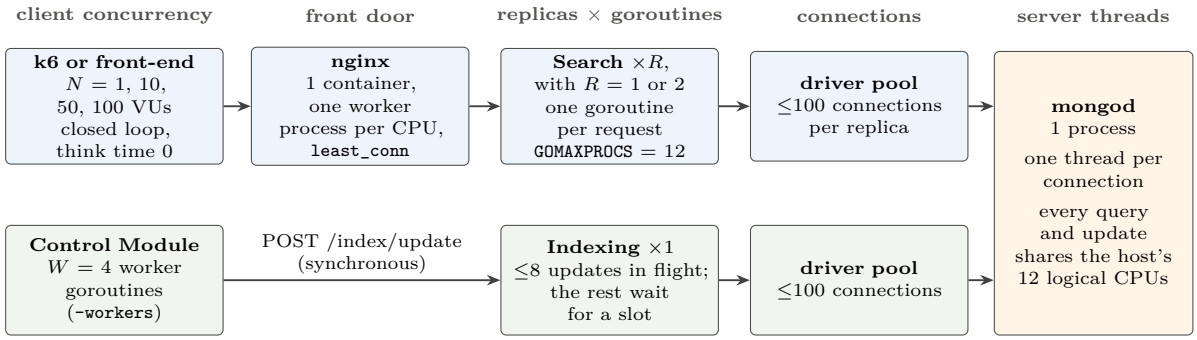


Figure 3: Levels of concurrency on the read path (top) and on the indexing write path (bottom). Client concurrency is set by the k6 virtual users (VUs) or by the Control Module's workers; replication multiplies the Search processes behind nginx; inside every Go process, `net/http` serves each request on its own goroutine, which the runtime multiplexes onto `GOMAXPROCS` operating-system threads. Every path ends in the same `mongod` process.

Distribution of requests among replicas. nginx re-resolves the compose service name `search` every 5 s and sends each request to the replica with the fewest requests in flight (`least_conn`), so replicas can be added without restarting the proxy; Section 6.6 justifies these settings.

Threads inside a process. Every service is a Go process that serves each request on its own goroutine (Figure 3). A goroutine blocked on network input or output (a MongoDB round trip, a Gutenberg download) is parked by the runtime and holds no operating-system thread. Concurrency that would overload a shared resource (downloads, index updates, Control workers) is bounded explicitly with buffered channels used as semaphores, and shared in-process state is protected by mutexes (Section 6.5).

Concurrent access to shared storage. Separate processes never share memory: they share the datalake, the control-state volume and the MongoDB server, and the rules that make their concurrent access safe are the shared-state contracts of Table 6.

3.4 Shared storage

Datalake. The named volume `datalake`, shared by the Ingestion and Indexing services, keeps the batch-based layout selected in Stage 1: each book has a folder `batch_<low>_<high>/<book_id>/` that holds `<book_id>_body.txt` and `<book_id>_header.txt`, with 1,000 consecutive IDs per batch folder, so book 1342 lives in `batch_1000_1999/1342/`.

Datamart. The MongoDB database (`boogle` by default; `boogle_bench` for the benchmarks) holds three collections, described in Table 2. Both services call an idempotent `EnsureIndexes` at start-up, so a fresh server needs no manual set-up.

Table 2: Collections of the MongoDB datamart.

Collection	Document	Secondary indexes
`books`	`{_id: book_id, title, author, language, publication_year, release_year, status}`; `status` is `pending` or `published`	`{author, language, publication_year}`, `{language}`, `{publication_year}`
`inverted_index`	`{_id: term, postings: [{book_id, tf}]}`, one document per normalised term, postings sorted by `book_id` without duplicates	`{postings.book_id}` (multikey)
`locks`	`{_id: "rebuild", owner, expires_at}`: the rebuild lease	—

Control state. The named volume `control-state` holds the Control Module's three append-only files, `downloaded_books.txt`, `indexed_books.txt` and `failed_books.txt`; their format and locking are specified in Table 6.

3.5 Single-host deployment

The whole system is described by one `docker-compose.yml`: the services `mongo`, `ingestion`, `indexing`, `search` (replica count from `SEARCH_REPLICAS`) and `nginx`, plus the Control Module, which is started on demand under the `tools` profile. One multi-stage `Dockerfile` builds the four statically linked Go binaries into images that run as an unprivileged user. Containers carry code only: the datalake, the control state and the datamart live in named volumes, so recreating a container never loses pipeline state. Services reach one another by compose service name, and every port, address and data path comes from configuration (Section 6.2). Health checks gate the start-up order: the Indexing and Search services wait for a healthy MongoDB, nginx for a healthy Search replica, and the Control Module for healthy Ingestion and Indexing services.

Every process, including the MongoDB server, the reverse proxy and, during the benchmarks, the k6 load generator and the resource sampler, runs on the one host described in Table 12. No container had CPU or memory limits in the official session, so all processes competed for the same 12 logical CPUs and 9.7 GiB of memory.

4 API Design and Contracts

4.1 Consumers and use cases

The interfaces serve three kinds of consumer: the front-end; the Control Module, which consumes the internal APIs (use cases UC4–UC8); and operational components such as Docker health checks and the interactive documentation (UC9). Table 3 lists the operations of each. The front-end, a browser application or an HTTP client such as k6 or `curl` exercising the same contract during Stage 2, has three use cases:

- *UC1*: search books by one or more keywords, with optional author, language and publication-year filters and paginated results;
- *UC2*: open the detail page of one book;
- *UC3*: populate the filter controls with the values that actually occur in the catalogue.

The front-end never reads files, never connects to MongoDB and does not need to know how the services are coordinated: its contract describes application data only, and ingestion commands, index rebuilds and datalake paths do not appear in it.

4.2 Endpoint inventory

Table 3 is the consolidated inventory, grouped by consumer. Every operation is *synchronous*: the response is sent when the work has completed (or failed), never when it has merely been accepted. Every operation is also *idempotent*, so a caller may repeat it after a timeout without creating duplicates. Both properties are stated per operation in the specification through the vendor extensions `x-completion` and `x-idempotent`.

4.3 Front-end contract

GET /search. The required search operation and all its filter combinations are variants of this single endpoint. Table 4 specifies its parameters. Several design rules complete the contract.

- **Term semantics.** `q` is split exactly like book text: runs of the letters a–z, case-insensitive, so digits, spaces and punctuation separate words (`Moby-Dick` searches `moby` AND `dick`). Repeated words count once. Stop words (*the*, *and*, *of*, ...), single letters and Roman numerals are never indexed and are ignored, so a `q` made only of them matches nothing and returns 200 with an empty page. A book matches only if it contains *every* remaining term (AND). The Search Service reuses the Indexing Service's tokenizer, so a query word and a body word always normalise alike.
- **Filters** combine with AND. `author` is a case-insensitive substring match, `language` an exact, case-insensitive ISO 639 code that matches any code of a multilingual book (stored as `"en,fr"`), and `year` an exact `publication_year`. Empty filter values are treated as absent, and only applied filters are echoed in `filters`.

Table 3: Endpoint inventory. Front-end routes are published by nginx on port 7003; internal routes are served on ports 7001 (Ingestion) and 7002 (Indexing). Error codes and their meaning are listed in Table 5.

Method and route	Provider → consumer	Purpose	Status codes
Front-end-facing (public, through nginx :7003)			
`GET /search`	Search → front-end	UC1: keyword search with optional filters, ranking and pagination	200, 400, 500
`GET /books/{book_id}`	Search → front-end	UC2: metadata of one published book	200, 400, 404, 500
`GET /search/filters`	Search → front-end	UC3: distinct languages and publication years of published books	200, 500
`OPTIONS` (any route)	Search → browser	CORS preflight of a cross-origin page	204, 403
Internal (consumer: Control Module)			
`POST /ingest/{book_id}`	Ingestion → Control	UC4: download, split and store one book	200, 400, 404, 500, 503
`GET /ingest/status/{book_id}`	Ingestion → Control	UC5: is the book complete in the datalake?	200, 400, 404
`GET /ingest/list`	Ingestion → Control	UC6: complete books, in ascending order (`-discover`)	200, 500
`POST /index/update/{book_id}`	Indexing → Control	UC7: index one book and publish it	200, 400, 404, 500
`POST /index/rebuild`	Indexing → Control (admin.)	UC8: rebuild the datamart from the whole datalake	200, 409, 500, 503
Operational			
`GET /index/status`	Indexing → Control, k6, Docker	Indexing statistics; readiness probe; corpus guard of the search benchmark	200
Health checks	every service → Docker	`GET /ingest/list`, `GET /index/status`, `GET /search/filters` (replicas and nginx), `mongosh` ping	200
`GET /books/{book_id}`	Search → Control	`-verify` mode: reconcile `indexed_books.txt` with the datamart	200, 404
`GET /swagger/index.html`	every service → developers	UC9: interactive OpenAPI documentation (`/swagger/doc.json`)	200

- **Ordering.** Results are ranked by relevance, defined as the sum of the query terms' frequencies in the book, in descending order, with ties broken by ascending `book_id`, so pagination is deterministic.
- **Pagination.** `total` counts every match across all pages, whereas `results` holds at most `limit` books starting at `offset`. A valid query without matches, or an `offset` past the last match, returns 200 with `results: []` and the true `total`.
- **Limits.** At most 10 distinct terms per query and 100 results per page are accepted.

Table 4: Parameters of `GET /search`. Violations are answered with `400 invalid_parameter` (or `missing_parameter` for an absent `q`) and a message naming the parameter.

Parameter	Type	Required	Default	Validation and semantics
`q`	string	yes	—	At least one word of letters a–z and at most 10 distinct terms; AND of its terms
`author`	string	no	none	Case-insensitive substring of the author
`language`	string	no	none	ISO 639 code, case-insensitive, any code of the book
`year`	integer	no	none	1–9999; exact publication year
`offset`	integer	no	0	0–$2^{31}-1$; matches skipped before the page
`limit`	integer	no	20	1–100; maximum results in the page

A representative exchange, taken verbatim from the validation transcript (check S5 in Table 7), shows the response schema:

```
GET /search?q=adventure&author=austen&language=en&limit=5          -> 200 OK
{"query":"adventure","filters":{"author":"austen","language":"en"},
 "total":1,"offset":0,"limit":5,
 "results":[{"book_id":1342,"title":"Pride and Prejudice",
             "author":"Jane Austen","language":"en","year":0}]}
```

GET /books/{book_id} and GET /search/filters. These two routes serve UC2 and UC3. `GET /books/{book_id}` returns the same record as one search result, and only for a *published* book: a book still being indexed

is as invisible here as in search. `GET /search/filters` returns `{"languages": [...], "years": [...]}`, computed from published books only.

4.4 Internal contracts

Ingestion. `POST /ingest/{book_id}` answers `downloaded` once both files are stored, or `available` without downloading when the book was already complete; a failure is either permanent (`404`) or transient (`503`), as listed in Table 5. Concurrent requests for the same book join one shared download, so the endpoint is safe under duplicate submissions.

Indexing. `POST /index/update/{book_id}` answers `updated` once the book's metadata and postings are in the *published* index, visible to every request that starts afterwards. Re-indexing replaces the book's metadata and postings, including terms the new version no longer contains, so repeating the call leaves the index exactly as after the first one; an update that arrives during a rebuild is not blocked but replayed on the rebuilt index. `POST /index/rebuild` builds a new catalogue and index aside and publishes them in one step, so search keeps reading the previous complete index until the swap, and also when the rebuild fails (Table 5). `GET /index/status` reports the published books, the time of the last change and the document bytes of the datamart (`dbStats.dataSize`). The literal request and response bodies of every internal operation are recorded in `docs/api_inventory.md` and the service specifications under `api/`.

Timeouts and retries expected of the caller. The specification states, per operation, the timeout and failure behaviour expected of the caller, which the Control Module implements (Section 5.5). A call to `POST /ingest` may take up to 150 s, above the service's own 2-minute budget, so the service always answers before the caller gives up. Indexing, status and readiness calls are bounded at 30 s. Transient failures (transport errors, timeouts, `408`, `429`, `5xx`) are retried three times with a doubling backoff that starts at 1 s, while permanent ones (other `4xx`) are recorded once and not retried. A client that disconnects cancels only its own index update, because the request context is propagated down to the MongoDB writes, whereas a shared download continues for the other callers that joined it.

4.5 Error model and status codes

Every service answers errors with the same JSON body, `{"error": "<code>", "message": "<human-readable explanation>"}`, where `error` is a stable machine-readable code and `message` is meant for logs and developers; Table 5 lists every code.

Table 5: Error codes shared by the three APIs.

Code	HTTP	Raised by	Meaning and expected caller action
`missing_parameter`	400	`GET /search`	`q` is absent; fix the request
`invalid_parameter`	400	`GET /search`	`q` has no word or more than 10 distinct terms, or `year`, `offset`, `limit` out of range
`invalid_book_id`	400	every `{book_id}` route	Path parameter is not a positive integer
`book_not_found`	404	`GET /books`, `GET /ingest/status`, `POST /index/update`	No published book (search) or no complete datalake copy (internal); ingest first, then retry
`book_unavailable`	404	`POST /ingest`	Gutenberg has no such book or no START/END markers; permanent, record the failure
`origin_not_allowed`	403	CORS preflight	Origin outside `CORS_ALLOWED_ORIGIN`
`rebuild_in_progress`	409	`POST /index/rebuild`	Another replica holds the lease; poll `GET /index/status` instead
`internal_error`	500	any	Storage failure; retry with backoff
`download_failed`	503	`POST /ingest`	Transient Gutenberg failure or budget exhausted; retry with backoff
`rebuild_interrupted`	503	`POST /index/rebuild`	Budget ran out or caller left; previous index still published; retry later

4.6 Non-HTTP contracts: shared state

Not every interaction is an HTTP call. The coordination of Section 3.2 also relies on shared state whose layout, ownership and synchronisation rules form part of the contract (Table 6). They agree with Figures 1 and 2.

Table 6: Shared-state contracts: what each record means, who may write it and how concurrent access is synchronised.

Contract	Location	Writer / readers	Synchronisation	Semantics
Datalake completeness	datalake volume	Ingestion / Ingestion, Indexing	Atomic replace per file (temporary file, `fsync`, rename); body before header	A book exists for readers if and only if both files exist
Control-state files	control-state volume	Control Module only	Exclusive `flock` to append, shared `flock` to read	One book ID per line; `failed_books.txt` holds `<id><TAB><reason>`
Publication state	`books.status`	Indexing / Search	Atomic per-document update; ordered writes (Figure 2)	Search returns `published` books only
Rebuild lease	`locks`, `_id: "rebuild"`	Indexing replicas	Claimed with `findOneAndUpdate`; 2-minute expiry, renewed every 30 s; taken over once expired	One rebuild at a time across replicas; the loser gets 409
Rebuild publication	`*_rebuild` collections	Lease holder / Search	`renameCollection` with `dropTarget`, index first, then books	Readers see the old or the new complete datamart

4.7 Exposure and cross-origin access

The front-end reaches the Search API through one entry point, nginx on port 7003, whatever the number of replicas. For a browser page served from another origin, the Search Service applies CORS to every route: the allowed origins come from `CORS_ALLOWED_ORIGIN` (`*` by default, safe for a read-only API without credentials, or an explicit list), and a preflight is answered with `204` or `403` (check X4 in Table 7). The internal ports 7001 and 7002 are also published on the host, for the Control Module and the Swagger pages, but the front-end contract never uses them.

4.8 Specification and validation evidence

Versioned specification. Each service carries a versioned OpenAPI 2.0 (Swagger) document, version 1.0.0, under `api/ingestion/`, `api/indexing/` and `api/search/` (`swagger.yaml` and `swagger.json`), published with the code at https://github.com/bigdata-devs/stage_2/tree/develop/api. The documents are generated with `swaggo/swag` v1.16.6 from annotations next to the handlers (Section 6.3), and each running service serves its own document as an interactive page at `/swagger/index.html`. Beyond schemas, parameters, defaults and status codes, every operation records its provider, consumers and audience (`x-provider`, `x-consumers`, `x-audience`), the completion and idempotency extensions of Section 4.2, and the timeouts and retries expected of the caller.

API validation script (19/19). `scripts/api_validation.sh` is the runnable check of the front-end contract. It seeds four fixture books (82, 84, 1341 and 1342) through the stack's own `POST /ingest` and `POST /index/update` routes, which exercises the internal contracts on the way, then sends nineteen requests to nginx on port 7003 and asserts status codes, error codes and response invariants that depend only on the fixtures, never on corpus-dependent totals. It exits with status 1 on any failure, so it doubles as a regression test, and `--record` regenerates `docs/evidence/api_validation_results.md` with the literal response body of every check. The recorded run of 31 October 2026 passed **19 of 19** checks (Table 7).

The run used local processes and MongoDB 8.3.7 at commit `c2cb456`, which predates the fallback of `publication_year` to the release year, so check S4 verified the *contract* of the year filter (applied, echoed, never violated) on fixtures without a known year (Section 7.9).

Internal interactions and further evidence. The internal HTTP contracts are exercised end to end by two scripts whose transcripts are committed. `scripts/e2e_control.sh` checks orchestration of real Gutenberg books, restart semantics (a replayed run skips finished books and leaves the state files intact), permanent-failure recording and discovery mode. `scripts/e2e_compose.sh` checks the Docker deployment in eight steps: health of every container, orchestration, idempotent replay, the front-facing routes, the fail-fast refusal of the Control Module while ingestion is down, recovery after relaunch, persistence of datalake and datamart across container recreation, and teardown; all pass. The cross-replica rebuild lease was verified live (`docs/evidence/rebuild_replica.md`, Section 7.2). Under load, the k6 checks *page respects limit* and *pagination is echoed* held for 100% of the 65,950 successful queries of the two search runs. The unit and integration tests are summarised

Table 7: The 19 checks of `scripts/api_validation.sh` and their recorded outcome (`docs/evidence/api_validation_results.md`). Every check passed.

ID	Case	Request	Expected (and observed)
S1	Keyword search	`/search?q=frankenstein`	200; book 84 present
S2	Author filter	`q=adventure&author=Austen`	200; 1342 present; every author contains "austen"
S3	Language filter	`q=adventure&language=EN`	200; filter echoed as `en`; every result in `en`
S4	Year filter	`q=adventure&year=1818`	200; filter echoed; no result breaks it
S5	Combined filters	`q=adventure&author=austen&language=en&limit=5`	200; 1342 present; 82 and 84 absent
S6	No matches	`q=quidditch`	200; `total` 0; `results` []
S7	Offset past the end	`q=adventure&offset=1000000`	200; `results` []; `total` still 3
V1	Missing `q`	`/search`	400 `missing_parameter`
V2	`limit`=0	`q=adventure&limit=0`	400 `invalid_parameter` naming `limit`
V3	`limit`=101	`q=adventure&limit=101`	400 `invalid_parameter` naming `limit`
V4	`year`=abc	`q=adventure&year=abc`	400 `invalid_parameter` naming `year`
B1	Book by ID	`/books/84`	200; metadata of book 84
B2	Unknown book	`/books/99999999`	404 `book_not_found`
B3	Malformed ID	`/books/abc`	400 `invalid_book_id`
F1	Filter values	`/search/filters`	200; `languages` includes `en`; `years` is an array
X1	Multi-word AND	`q=modern+prometheus`	200; only book 84
X2	No word in `q`	`q=1813`	400 `invalid_parameter` naming `q`
X3	Stop words only	`q=the`	200; `total` 0
X4	CORS preflight	`OPTIONS /search?q=whale` from `localhost:5173`	204 with `Access-Control-Allow-Origin`

in Section 5.7.

5 Microservice Implementation

5.1 Technology stack and code organisation

The assignment's reference template is a Java 17 service built with Javalin, Gson and Maven and benchmarked with JMH. The group implemented the services in **Go** instead, the language selected by the Stage 1 measurements (highest datalake write throughput of the four implementations compared, and fastest index builds from 500 books upward), and every element of the template has a standard counterpart. The `net/http` server with the method-aware `http.ServeMux` of Go 1.22, which answers `405` to an undeclared method, replaces Javalin without any third-party framework; `encoding/json`, with struct tags that fix the field names of the contract, replaces Gson; one Go module (`boogle-engine`, Go 1.25) with four entry points under `cmd/` replaces Maven; and `testing.B` with `benchstat` replaces JMH, with the benchmarks kept in `_test.go` files and therefore out of the service binaries. The datamart is accessed through the official MongoDB Go driver (v2.9.2), the API documentation is generated with `swaggo/swag`, and logging uses the structured `log/slog` logger.

Repository layout. The Go code follows a **hexagonal (ports and adapters) structure**: each use case lives in a *core* package that declares, as Go interfaces, the operations it offers (*driving* ports such as `Searcher`) and the operations it needs (*driven* ports such as `search.Index`). Source dependencies point only into the core, and every adapter asserts at compile time the ports it implements, so the use cases are tested with in-memory fakes and moving a store to Stage 3 infrastructure means writing a new adapter rather than changing a service. The packages are grouped as follows (the full table is in `docs/architecture.md`):

- *Cores* (no HTTP, MongoDB or filesystem code): `internal/models` (domain and shared errors), `internal/indexer` (tokenizer and metadata parser), and the use cases in `internal/ingestion`, `internal/indexing`, `internal/search` and `internal/control`.
- *Driving adapters* (inbound HTTP): `internal/api` (shared kit) and the three service packages under it (`ingestionapi`, `indexingapi`, `searchapi`) with their handlers, DTOs and CORS.
- *Driven adapters* (outbound): `internal/crawler`, `internal/datalake`, `internal/mongodb`, `internal/control/httpclient` and `internal/localstate` (the `flock`-protected state files).
- *Composition roots, specifications and deployment*: `cmd/*`, `api/*`, `deploy/nginx`, `docker-compose*.yml`, `test/load`, `scripts` and `docs`.

The services share one skeleton: `main`, the composition root, parses flags and environment variables, builds the adapters and injects them into the core constructors (constructor injection), mounts the handlers on a `ServeMux`, starts an `http.Server` whose read, header, write and idle timeouts are bounded, and shuts down gracefully on `SIGINT` or `SIGTERM` (30 s grace for Ingestion and Indexing, 10 s for Search).

5.2 Ingestion Service

A request for `POST /ingest/{book_id}` flows from the handler into the **ingestion core** (`ingestion.Service`) and from there to two driven adapters. The core implements single-flight per book: concurrent requests for the same ID join one shared ingestion, tracked in an in-flight map guarded by a mutex. Each ingestion runs detached from the requests that wait for it, under its own time budget, so a caller that disconnects cancels neither the shared work nor the other callers. If the datalake already holds the complete book, the core returns at once; otherwise it waits for a download slot (a buffered channel) and the **Gutenberg client** downloads `https://www.gutenberg.org/cache/epub/<id>/pg<id>.txt` through one shared `http.Client` with connection reuse, retrying only the errors it classifies as transient. The **splitter** then separates header and body at the START/END markers, and the **datalake** adapter stores both files with the atomic protocol of Table 6, so a crash at any point leaves at most a stray temporary file, never a truncated book. The budget, slot count and retry settings are listed in Table 11.

5.3 Indexing Service

One update. `POST /index/update/{book_id}` first acquires an update slot (Table 11); a request that finds them all busy waits until a slot frees or its caller disconnects, so a burst of updates cannot exhaust the datamart connection pool. The **indexing core** (`indexing.Service`) then reads the book from the datalake through its `BookSource` port, refusing an incomplete one, extracts the metadata from the header (title, author, language mapped to ISO 639 codes, original publication year falling back to the Gutenberg release year), and turns the body into terms with the Stage 1 tokenizer, whose rules are those of the search contract (Section 4.3). The MongoDB writer, which implements the core's `IndexWriter` port, stores the book in four ordered, idempotent steps (Figure 2):

1. replace the book's metadata record with `status: pending`;
2. remove the book from every posting list it appeared in (`$pull`, bounded by the multikey index on `postings.book_id`), so a re-indexed book forgets terms it no longer contains;
3. upsert one posting per distinct term in unordered bulk writes of at most 1,000 operations. Each upsert is an update pipeline that filters out any previous entry of the book (`$filter`), appends the fresh `{book_id, tf}` pair (`$setUnion`) and re-sorts the list by book ID (`$sortArray`), so replays converge to the same document;
4. set `status: published`, the moment the book becomes searchable.

Rebuild and publication. `POST /index/rebuild` first claims the cross-replica lease of Table 6, which a background goroutine renews while the rebuild runs. The holder streams the datalake one book at a time into the temporary collections `books_rebuild` and `inverted_index_rebuild`, so its memory does not grow with the datalake, creates their indexes, and publishes them with two `renameCollection` commands (index first, then books); search keeps reading the previous complete datamart until the swap. Updates that arrive during a rebuild are not blocked: an atomic flag, read *before* each datamart write, makes the indexing core record the book in a mutex-protected replay list, and the list is re-applied to the fresh datamart after the swap, so no concurrent update is lost. Recording before writing guarantees that a swap can never overtake a book that is still being written.

Status. `GET /index/status` counts the published books and reads the datamart size from `dbStats`. It never fails: a measurement that cannot be taken is reported as zero and logged, because the endpoint doubles as a readiness probe.

5.4 Search Service

The handler reads the query string and passes it to the **search core** (`search.Service`), which validates it (Table 4) and splits `q` into at most ten distinct normalised terms with the indexer's tokenizer. When every word of `q` is one the index never stores, the core answers an empty page without querying MongoDB. Otherwise it calls its `Index` port, which the MongoDB adapter implements with **one aggregation** on `inverted_index`,

so a request costs a single round trip (keeping matching and ranking in the aggregation is a deliberate choice, discussed in Section 6.4):

1. `$match` the term documents (`_id $in` terms) and `$unwind` their postings;
2. `$group` the postings by book, collecting the matched terms and summing their frequencies into a score, then keep only the books that matched every term (the AND);
3. `$lookup` each surviving book in `books`, requiring `status: published` and applying the author, language and year filters inside the lookup;
4. `$facet` into a total count and a page sorted by score (descending) and book ID (ascending), with `$skip` and `$limit`.

Intersecting the postings before the join means that every additional term shrinks the join instead of growing it. `GET /books/{id}` and `GET /search/filters` go through the catalog use case (`search.CatalogService`) and read the same datamart through the same stateless reader, restricted to published books. The process holds no mutable state beyond the driver's connection pool, which is why replicas are interchangeable and why nginx may route any request to any of them. At start-up the service refuses to run if MongoDB is unreachable (fail fast), instead of answering `500` to every request.

5.5 Control Module

The Control Module is a command-line tool (`cmd/control`) that selects candidate books (`-ids`, `-from`/`-to`, or `-discover` from `GET /ingest/list`), loads the control state, and refuses to start unless both services answer their readiness routes within 10 s. A pool of `-workers` goroutines (4 by default) then processes the candidates. A mutex-protected **book selector** hands each candidate to exactly one worker; the worker skips books already indexed or failed, ensures ingestion, records it, ensures indexing and records it, applying the retry policy of Section 4.4 to each step. A book interrupted by `SIGINT` or `SIGTERM` is left pending and claimed again by the next run. Two further modes support operations: `-retry-failed` atomically empties `failed_books.txt` and re-enqueues its books, and `-verify` reconciles `indexed_books.txt` with the datamart through `GET /books/{id}`. Every finished step is logged with its duration, and the run summary reports throughput and the time spent in ingestion and indexing steps, which provides the coordination-cost measurements of Section 7.7.

5.6 Readiness, task ownership, retries, idempotency and publication

Table 8 gathers, for each concern the assignment names, the mechanism that implements it.

Table 8: How the implementation handles readiness, ownership, retries, idempotency and publication.

Concern	Mechanism	Where
Book ready for indexing	Datalake completeness contract (Table 6); an incomplete book is refused	`internal/datalake`, indexing handler
Service readiness	Docker health checks gate start-up; Control readiness probe (10 s); Indexing and Search refuse to start without MongoDB	`docker-compose.yml`, `cmd/*`
Task ownership	Selector gives each book to one worker per run; one Control instance at a time; single-flight per book in Ingestion; rebuild lease across replicas	`internal/control`, `internal/ingestion`, `rebuild_lock.go`
Retries	Control: 3 attempts with 1 s doubling backoff for transient failures only (Section 4.4); Ingestion downloads: same policy	`orchestrator.go`, `internal/ingestion`
Idempotency	Ingestion answers `available` for complete books; index updates replace metadata and converge postings; control records skip known IDs	`ingestion`, `mongodb`, `control`
Publication	`status: published` is the last write of an update; rebuild swaps complete collections; updates during a rebuild are replayed	`book_indexer.go`, `rebuild.go`
Crash between step and record	Book not recorded, so the next run replays the idempotent call	`orchestrator.go`

5.7 Testing

The repository contains unit tests for every package and integration tests that run against a live MongoDB server. These tests skip quickly when no server is reachable, and a recorded run of `go test -count=1 -v ./...` against the `mongo:7` container reported 365 passing tests, none skipped and none failing (`docs/evidence/tests.txt`). The suites cover the orchestration loop against fake services (happy path, resumption, retries, permanent failures, shutdown, concurrent workers), the HTTP clients against `httptest` servers, every handler's contract including error codes and CORS, the write path's idempotency and the visibility of pending books, the

rebuild and its lease, and the search aggregation's ranking, filters and pagination. The micro-benchmarks of Section 7.6 live in the same `_test.go` files.

6 Design Decisions

6.1 Coordination mechanism

The assignment admits four coordination strategies. Table 9 compares them against the requirements that matter for this pipeline: a book must be indexed only after its ingestion has completed, an update must be published before search can see it, and repeated work must not corrupt the datamart.

Table 9: Coordination strategies considered. The chosen design is classical orchestration, combined with persistent markers, a publication state and a lease kept in shared storage.

Criterion	**Classical orchestration** (chosen)	Choreography with a broker	Time-based coordination	Shared memory
Ingest-before-index	Explicit: indexing is called only after a `2xx` from ingestion	Implicit: a `BookDownloaded` event triggers indexing	Requires readiness markers; a fixed delay proves nothing	Atomic state transitions in a shared region
Completion signal	Synchronous response: `200` means done	Acknowledgements, at-least-once delivery	Persistent markers, polled	Flags guarded by inter-process locks
Extra components	None beyond a command-line tool	A broker process with queues and dead-letter handling	A scheduler; overlap and missed-run handling	Memory-mapped region, inter-process synchronisation
Failure handling	One retry policy; durable progress records	Redelivery, duplicate suppression, ordering rules	Late or overlapping runs	Recovery of a region left locked by a crashed process
Observability	One log with per-step durations	Requires distributed tracing	Run history	Hard to inspect
Stage 3	HTTP works across machines; the control state must move to shared storage	Natural fit for a cluster	Works with distributed markers	Must be replaced: single host only

Decision and rationale. Classical orchestration makes the one hard dependency of the pipeline explicit in code rather than emergent from message timing, and its synchronous answers give the clearest definition of "done": a book is recorded as indexed only after it has been *published*. The alternatives buy nothing on one host. A broker would add a stateful process on a host whose CPU is already the scarce resource (Section 7.4), while still requiring the idempotent consumers our services already are; time-based runs would only add latency on top of the readiness markers needed anyway; and shared memory is unsafe across containers and must be replaced in Stage 3. Orchestration alone is not enough, because an operator or a second Indexing replica can also touch shared data, so correctness also rests on the shared-state mechanisms of Section 3.2 and on idempotent operations.

Trade-offs accepted. The Control Module is a single point of coordination: while it is down, no new book enters the pipeline, although search keeps serving the published index. It is stateless across restarts, because its progress lives in the control files, so a restarted run resumes. Exactly-once claiming holds within one run; running two Control instances concurrently is excluded by operating procedure rather than enforced, a limitation addressed in Section 8.3. Synchronous calls hold a worker for the whole duration of an index update (1–7 s in Section 7.7), so the worker count is the throughput knob of the pipeline.

6.2 Single-machine deployment

Stage 2 runs every process on one machine, and the deployment is designed so that the experiment can be reproduced and later moved without code changes. Docker Compose was chosen because a single file then reproduces the whole topology of Section 3.5, start-up order and health checks included. Every address, path, replica count and concurrency setting is configurable rather than hard-coded, and no `localhost` address appears in service code. Ports come from `INGESTION_PORT`, `INDEXING_PORT`, `SEARCH_PORT` and `MONGO_PORT` (or each service's `-addr`); endpoints from `INGESTION_URL`, `INDEXING_URL`, `SEARCH_URL`, `MONGO_URI` and `MONGO_DATABASE`; data paths from `-datalake`, `-batch-size` and `CONTROL_STATE_DIR`; the Search replica count from `SEARCH_REPLICAS` (or `--scale search=N`, or `docker-compose.replicas.yml` for two replicas); and allowed origins from `CORS_ALLOWED_ORIGIN`.

The concurrency and time settings of each process are in Table 11. Because replicas, threads, the datamart server and the load generator share one CPU, memory and disk, adding workers can increase contention rather than throughput, which is what Section 7 measures.

6.3 API design

- **Synchronous, idempotent internal operations.** Ingestion and indexing of one book take from milliseconds to a few seconds, well within an HTTP timeout, so a synchronous response whose `200` means "completed and published" removes the need for job identifiers and status polling. Making every `POST` idempotent turns retries after timeouts into a safe default instead of a hazard. The one long operation, a full rebuild, stays synchronous but is guarded by a lease and a budget, and it can be observed through `GET /index/status`.
- **Bounded requests.** The limits of at most 100 results per page and 10 terms per query cap the cost of any single request, which is a precondition for predictable latency under concurrency.
- **Offset pagination with deterministic order.** Offset and limit are stateless and simple for a front-end, and the deterministic order of Section 4.3 keeps pages from overlapping. Deep offsets cost the datamart work proportional to the offset, a known limit of this scheme (row 5 of Table 22).
- **One error schema, stable codes.** Machine-readable codes let callers distinguish permanent from transient failures without parsing messages, which is what the Control Module's retry policy relies on.
- **Specification generated from code.** Annotations live next to the handlers and the specification is regenerated with `go generate`, so contract and implementation are reviewed in the same commit; the validation script then checks that the running service matches the contract.

6.4 Storage selection

Datalake on a shared volume. The batch-based layout is kept because it matched the book-based layout on lookup time in Stage 1 with far fewer directories. On one host a named volume is the simplest shared store, and atomic replacement of files gives crash safety without a database. Its limitation is locality: the volume exists on one machine only (Section 8.3).

MongoDB as the datamart. Stage 2 relies on every requirement for which MongoDB was provisionally selected (Section 2). Separate processes, including several Search replicas, share one index over the network instead of each holding a copy. Per-document atomic updates and idempotent upserts let concurrent writers converge, secondary indexes back the metadata filters, `renameCollection` publishes a rebuild atomically, and `findOneAndUpdate` provides a cross-process lease. The file-based indexes of Stage 1 offer none of these properties: they would require a single writer, file locking across processes, and an in-memory copy of the index inside every Search replica.

Query evaluation stays inside the datamart. In the hexagonal structure (Section 5.1) the search core decides what a query means but not how it is evaluated. Keeping AND matching and relevance ranking inside the MongoDB aggregation of Section 5.4, behind the core's `Index` port, is a deliberate performance choice rather than an omission. Evaluated on the server, a query costs one round trip and returns one page of metadata together with the total count; evaluated in the Search Service, it would transfer every posting of every query term on each request, and the posting list of a common term can span most of the corpus. The price is that these rules belong to the adapter: they are documented as the contract of the port and pinned by the datamart's integration tests, so any future `Index` adapter must reproduce them.

Costs of the data model. Storing one document per term with its whole posting list makes reads simple but has two costs that the benchmarks expose. The first is *write amplification*: re-indexing book 1342 rewrites the posting arrays of its 6,576 distinct terms twice, once to remove the old entry and once to upsert the new one. The second is a *size ceiling*: a BSON document is limited to 16 MB, so the posting list of a very common term would hit the limit at a few hundred thousand books. Both costs are accepted for Stage 2 and addressed in the Stage 3 plan (Section 8.3).

6.5 Concurrency model and protection of shared state

A goroutine costs a few kilobytes of stack and one blocked on I/O holds no thread (Section 3.3), so request concurrency is cheap in Go, and the runtime default of one operating-system thread per logical CPU (`GOMAXPROCS`

= 12) was kept. The design decisions therefore concern *where to bound* concurrency and *how to protect* shared in-process state; Table 10 lists the mechanisms. As the assignment stresses, an in-process lock protects the goroutines of its own process only: wherever separate processes or replicas must agree, the rule is one of the shared-state contracts of Table 6, held in the file system or in MongoDB.

Table 10: In-process synchronisation mechanisms. Cross-process rules are the shared-state contracts of Table 6.

Mechanism	Where	Protects or bounds
Buffered channel (8 slots)	ingestion core	Concurrent Gutenberg downloads
In-flight map + `sync.Mutex`	ingestion core	One download per book at a time (single-flight)
Buffered channel (8 slots)	indexing handler	Concurrent index updates, hence load on MongoDB
`atomic.Bool` + `sync.Mutex`	indexing core	Rebuild-in-progress flag; replay list of concurrent updates
`sync.RWMutex`	control state	In-memory sets of downloaded, indexed and failed books (read-mostly: every step reads, transitions write)
`sync.Mutex`	book selector	Claim cursor, so that each book goes to one worker
`sync.WaitGroup`, `context`	orchestrator, all services	Worker lifecycle; cancellation and deadlines propagated to MongoDB

The read-write lock of the control state illustrates the design rule: every worker checks `IsIndexed` and `IsDownloaded` before each step, concurrently and under a shared lock, while the rarer transitions take the exclusive lock and append to the file under `flock`. Thread exhaustion is prevented by the bounded semaphores and by server timeouts: no request can pin a goroutine indefinitely, and a burst of index updates queues in the handler instead of opening unbounded connections to MongoDB. The driver's default pool of at most 100 connections per process is the last bound.

6.6 Replica counts, thread settings and ports

Table 11 records the configuration of every process, including the settings that came from framework or runtime defaults.

Table 11: Replicas, ports, concurrency settings and time limits per process, with the flag that sets each value (defaults; "tested" lists the replica counts measured in Section 7).

Process	Replicas (tested)	Port	Concurrency settings	Time limits
nginx	1	7003	`worker_processes auto` (one per CPU), `worker_connections 4096`, upstream keep-alive 64, `least_conn`	connect 2 s, read 20 s; retry on connection error only (2 tries)
Search	1 (1 and 2)	internal 7003	goroutine per request; `GOMAXPROCS` 12 (runtime default); driver pool $\leq$100	header 5 s, read 10 s, write 15 s, idle 60 s
Ingestion	1 (1)	7001	`-workers` 8 download slots; single-flight per book	`-ingest-timeout` 2 min; `-timeout` 30 s per download; `-attempts` 3, `-backoff` 1 s
Indexing	1 (1; 2 in the lease test)	7002	`-max-concurrent-updates` 8; one rebuild cluster-wide	`-rebuild-timeout` 10 min; write deadline 10 min + 10 s
Control	1, on demand	—	`-workers` 4	`-ingest-timeout` 150 s; `-timeout` 30 s; readiness 10 s
MongoDB	1	27017	one server thread per connection; WiredTiger defaults	—

The nginx front door. nginx publishes the Search API on a stable port, so clients, k6 and the validation script use `http://localhost:7003` whatever the number of replicas. Its configuration encodes four further decisions. First, `least_conn` sends each request to the replica with the fewest requests in flight, since query costs vary widely. Second, `resolve` with a 5 s DNS validity lets `--scale search=N` add or remove replicas without restarting the proxy. Third, `proxy_next_upstream error` retries a request on another replica only when the first cannot be *reached*; slow answers are never retried, because retrying them would double the load precisely when the system is saturated. Fourth, the 20 s read timeout exceeds the replicas' 15 s write timeout, so a slow search fails at the replica, which logs it, rather than at the proxy. Every response carries an `X-Upstream` header naming the replica that served it, which made the load split of the replica experiment observable.

6.7 Where replication and threading help, and where they add contention

The decisions above lead to expectations that Section 7 tests:

- **Stateless Search replicas** can be added freely and spread requests evenly. They raise throughput only if the Search process itself is the bottleneck; if the shared datamart or the host CPU saturates first, replicas merely move the same work into more processes, and each replica's connection pool adds server threads to an already busy `mongod`. On one host their certain benefit is availability: nginx routes around a replica that stops.
- **Threads for CPU-bound work**, such as tokenisation, metadata parsing and header/body splitting, should scale up to the number of physical cores (six on this host) and less beyond, where they share cores through simultaneous multithreading.
- **Concurrency for network-bound work**, such as downloads from Project Gutenberg, should scale almost linearly until the upstream server or the network limits it, because waiting goroutines cost no CPU.
- **Concurrent index updates** contend inside MongoDB: books share many terms, so concurrent updates rewrite the same term documents. More update slots or Indexing replicas would add contention rather than throughput, and multiple Indexing replicas would additionally require routing updates to the rebuild owner, because the replay list of concurrent updates is kept per process.

7 Benchmarks and Results

This section answers the six benchmarking questions of the assignment with measurements taken on one machine; Section A maps every table and figure to its raw evidence. In every figure, colour follows the component (■ coral MongoDB, ■ teal Search, ■ indigo Indexing, ■ blue nginx, ■ plum Ingestion), dashed lines are models or bounds, and dotted lines objectives or hard capacities.

7.1 Experimental setup

Host. All official measurements were taken in one session on 1 November 2026, at commit `ade7245`, on the machine described in Table 12; the only later code change moves the admission bound of index updates into the HTTP handler without changing its behaviour.

Table 12: Host and runtime of the official benchmark session.

Component	Value
CPU	Intel Core i7-10750H @ 2.60 GHz: 6 cores / 12 threads (12 logical CPUs visible to WSL2)
Memory	15.8 GB Windows host; 9.7 GiB for the WSL2 distribution that runs Docker (container limit 9.712 GiB)
Storage	Repository on Windows NTFS (1.9 TB); Docker volumes on the WSL2 ext4 root (1007 GB)
OS and power	WSL2, Linux 6.18; Windows "High performance" plan; machine idle on AC during the session
Containers	Docker Engine 29.1.3 (native in WSL2), Compose 2.40.3; no CPU or memory limits
Services	Go binaries on `alpine:3.21`; `GOMAXPROCS` = 12 (runtime default)
Datamart and proxy	MongoDB 7.0.45, a fresh `boogle_bench` database per k6 run; nginx 1.28
Load generation	k6 v2.3.0 on the same host; Go 1.27.2 for tests and micro-benchmarks

Experiments. Table 13 lists every experiment, its workload and its evidence files. The integration benchmarks drive the running services over HTTP with k6 in a *closed model*: each virtual user (VU) sends a request, waits for the response and immediately sends the next one (think time 0), so the number of VUs *is* the client concurrency. The load is stepped: for each level, k6 ramps up for 10 s and holds the level for 30 s, and only the hold phases enter the per-level results, so ramps never blur the steady state. Search requests follow a seeded mix (seed 2026, identical in every run): 81 real words and phrases in bands of document frequency, plus a few words that match no book and a lone stop word; one query in five is a two-word phrase, an author filter is added with probability 0.25, a language filter with 0.35 and a year filter with 0.10, and page sizes range from 10 to 100.

Metrics and measurement rules. *Throughput* X is the number of requests completed in a hold phase divided by its 30 s; for indexing, one successful update indexes one book, so requests per second equal books per second. *Latency* R is the response time of every successful request as measured by k6, summarised by mean, median and the 90th, 95th and 99th percentiles. *CPU* is the `docker stats` value expressed in logical cores (100% of one core = 1.0), so the host provides 12.0; memory is the container's resident memory. Each per-level resource value averages the 8 to 10 samples that fall inside the hold window, aligned with the k6 phases (Section A), and every configuration was run once (Section 7.9).

Table 13: Experiments, workloads and evidence. "Official" means the session of Table 12; the other two experiments are reported with their own context and are not compared numerically with the official ones.

Experiment	Data	Load and configuration	Evidence (`docs/evidence/`)
Search, 1 and 2 replicas (official)	300 published Gutenberg books, seeded with `seed_corpus.sh`	k6 `search_benchmark.js`, 1, 10, 50, 100 VUs; nginx in front of 1 or 2 replicas, nothing else changed	`search_benchmark_*_{1replica,2replicas}.*`
Indexing (official)	20 real books already indexed (steady-state re-indexing)	k6 `indexing_benchmark.js`, 1, 5, 10 VUs on `POST /index/update`	`indexing_benchmark_*.*`
Resources (official)	—	`docker stats` of every container, one sample about every 3 s	`search_resources*.csv`, `indexing_resources.csv`
Rebuild lease (official)	datalake of 383 books	concurrent `POST /index/rebuild` on two Indexing replicas	`rebuild_replica.md`
Micro-benchmarks (official)	383 real books; 300-book synthetic datamart; book 1342	`go test -bench`, 10 samples, `-cpu 1,2,4,8`	`microbenchmarks.txt`, `benchstat*.{txt,csv}`
Ingestion	100 popular Gutenberg IDs, cold datalake	1, 2, 4, 8 download workers; *other host*: Core Ultra 7 155H, 32 GB, Windows 11	`ingestion_throughput.md`
Coordination	100 IDs per run (1–100, . . . , 301–400)	Control Module with 1, 2, 4, 8 workers on the compose stack; *pre-official*, 29 Oct.	`cmd/control/results/`

Analytical lens: operational laws. Because the load is a closed system in steady state, three laws of operational analysis hold for every hold phase without any assumption about service-time distributions, and they turn the plots below into diagnoses:

- *Little's law*, $N = X\,(R+Z)$: with think time $Z = 0$, the mean response time must equal N/X. A mismatch would reveal failed or unaccounted work.
- *Utilisation law*, $U_k = X\,D_k$: the busy time of resource k is the throughput times the *service demand* D_k, the resource time one request consumes. Dividing the CPU of each container by the throughput therefore yields the CPU cost of one request in each process.
- *Asymptotic bounds*: $X(N) \le \min\{N/D,\ X_{\max}\}$ and $R(N) \ge \max\{D,\ N/X_{\max}\}$, where D is the response time of a lone request and $X_{\max} = m/D_{\text{cpu}}$ is the capacity of the bottleneck (m processors, D_{cpu} CPU seconds per request). The two bounds meet at the *knee* $N^* = X_{\max}D$, the concurrency beyond which extra clients can only queue.

7.2 Indexing throughput

Table 14 and Figure 4 report the steady-state indexing benchmark; Table 15 gives the resource samples behind Figure 4b.

Table 14: `POST /index/update` under concurrent load: throughput and latency per concurrency level (hold phases of 30 s, 20-book corpus, no errors). Speed-up and parallel efficiency are relative to one client. With 33 to 160 updates per level, the 99th percentile rests on few samples.

VUs	Updates	Books/s	Speed-up	Efficiency	Mean	Median	p90 (ms)	p95	p99	Max
1	33	1.10	1.00	100%	919	849	1,520	1,692	2,109	2,298
5	108	3.60	3.27	65%	1,370	1,335	2,535	2,904	3,410	3,452
10	160	5.33	4.85	48%	1,904	1,889	3,005	3,189	4,078	4,174

Table 15: Resource use during the indexing benchmark (mean over each hold phase; memory as mean and, in parentheses, maximum).

	CPU (logical cores)			CPU per book (core-s)		Memory (MiB)	
VUs	`mongod`	Indexing	Total (host)	`mongod`	Indexing	Indexing	`mongod`
1	0.96	0.16	1.12 (9%)	0.87	0.15	22 (26)	233 (234)
5	4.59	0.58	5.17 (43%)	1.27	0.16	64 (81)	240 (245)
10	7.31	0.84	8.15 (68%)	1.37	0.16	95 (115)	268 (398)

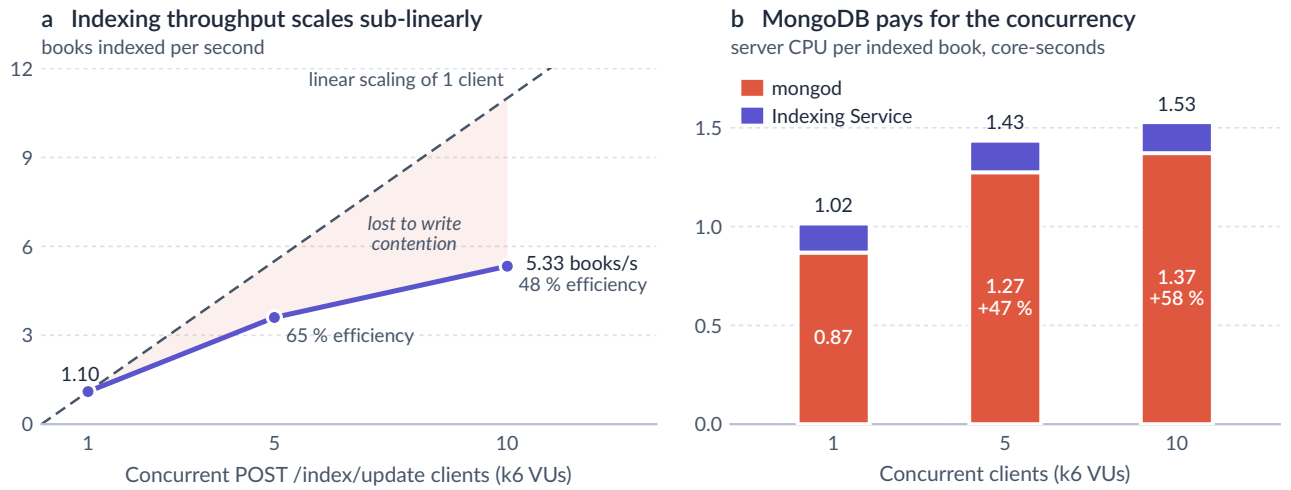


Figure 4: Indexing benchmark, `POST /index/update` on the 20-book corpus (official session, one Indexing replica, eight-slot admission bound). (a) Measured throughput against linear scaling of the single-client rate (dashed); the shaded area is the throughput lost relative to linear scaling, and the labels give the parallel efficiency. (b) CPU consumed per indexed book, from the `docker stats` samples divided by the throughput (utilisation law): the `mongod` share grows by 58% from one to ten clients while the Indexing Service's own cost stays flat.

Sub-linear scaling. Ten concurrent updates index 5.33 books/s, 4.85× the single-client rate of 1.10 books/s, so parallel efficiency falls from 65% at five clients to 48% at ten (Figure 4a). The gap to linear scaling is not lost work: all 423 timed updates succeeded, and the mean latency of every level agrees with Little's law, $R = N/X$ (0.91, 1.39 and 1.88 s predicted against 0.92, 1.37 and 1.90 s measured). The extra latency is therefore time spent waiting or contending. Part of it is designed: at ten clients the handler admits eight updates and the other two wait for a slot.

Contention, not capacity. A capacity limit and a contention limit look alike on a throughput curve, but the utilisation law separates them. If indexing were capacity-bound, the CPU cost of a book would stay constant and throughput would stop only when the host ran out of CPU. Neither happens. At ten clients the stack uses 8.15 of the 12 logical CPUs (68%), yet throughput has already flattened; and the cost of a book is not constant but rises from 1.02 to 1.53 core-seconds (Figure 4b). All of that growth is inside `mongod`, whose cost per book grows by 47% at five clients and 58% at ten, while the Indexing Service spends a flat 0.15–0.16 core-seconds per book on reading, tokenising and encoding the writes. Measured against the single-client cost, about one core-second in three that the stack consumes at ten clients is overhead of concurrency: at the single-client cost, the same 8.15 cores would have indexed 8.0 books/s instead of 5.33.

The mechanism is the write amplification of the data model (Section 6.4): each update rewrites thousands of posting arrays, and the books of the corpus share most of their common terms, so concurrent updates write the same term documents. Serialised document writes and retried write conflicts inside the storage engine are consistent with a per-book cost that grows with concurrency; the attribution remains a hypothesis, because `serverStatus` counters were not recorded (Section 7.9).

The write path in isolation. The micro-benchmark `IndexBookRealBook` re-indexes book 1342 through the real write path on an otherwise empty datamart: 636.9 ms per book (±2%, 95% confidence), for 6,576 distinct terms and 58,860 tokens, with 42.4 MiB and 444,400 allocations per call on the client side. Because every posting array holds a single entry in this benchmark, its cost is the per-term overhead of the update pipeline, not the length of the lists. Tokenising a book costs 13.7 ms on one thread (Table 19), about 2% of the write. Indexing time is therefore dominated by datamart writes, and tokenising faster would not help.

Full rebuild. In the rebuild-lease experiment, replica A rebuilt the 383-book datalake in 8 min 0.9 s (0.80 books/s through the single-writer rebuild path). Replica B's overlapping request was answered `409 rebuild_in_progress` in under a second, and its retry after the lease was released succeeded in 7 min 54.5 s (0.81 books/s).

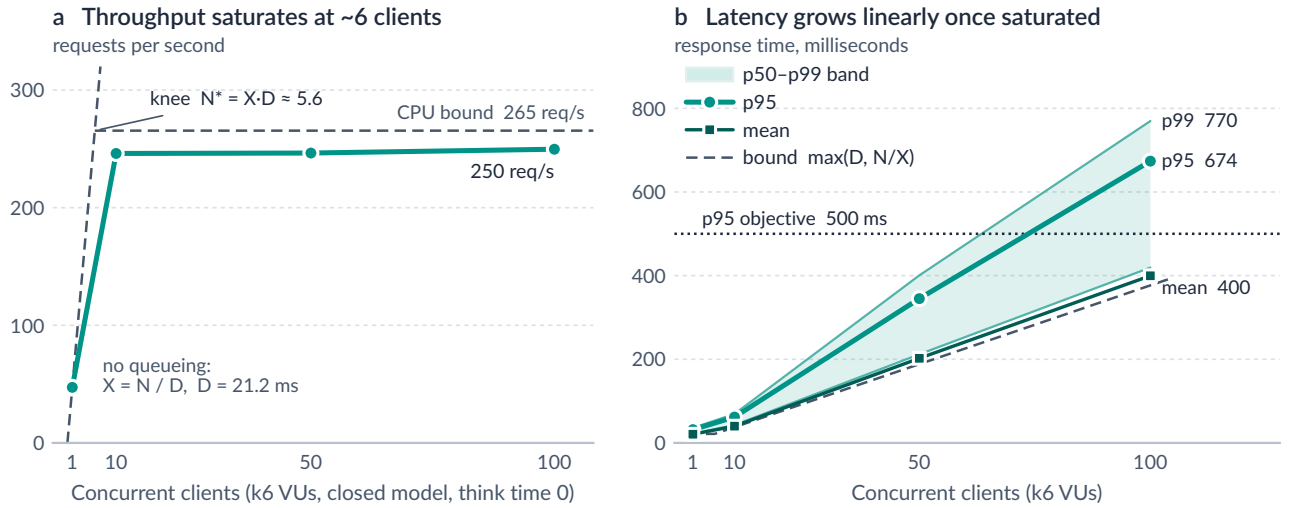


Figure 5: `GET /search` through nginx, one Search replica, 300-book datamart, 30 s hold per level. (a) Throughput against the asymptotic bounds: without queueing, $X = N/D$ with $D = 21.2$ ms, the response time of a lone query; the CPU bound $X_{\max} = 12/D_{\text{cpu}} = 265$ req/s follows from the 45.2 core-ms of server CPU that one query costs at saturation (Figure 6c). The bounds meet at the knee $N^* \approx 5.6$ clients. (b) Latency distribution: the band spans the median to the 99th percentile; the mean stays 6–7% above the lower bound $\max(D, N/X_{\max})$, and the 95th percentile crosses the 500 ms objective between 50 and 100 clients.

7.3 Query latency and concurrency

Figure 5 plots the search benchmark with one Search replica against the operational bounds of Section 7.1; Table 16 lists both replica configurations. Neither run produced an error: 33,131 and 32,819 successful queries, all of which passed the k6 checks on status, schema, page size and echoed pagination. About 30.5% of the responses were empty pages (no-match words, stop words, restrictive filters or offsets past the end), an intended part of the mix.

Table 16: `GET /search` through nginx with one and two Search replicas: steady-state throughput and latency per concurrency level (hold phases of 30 s, 300-book datamart, 0% errors). N/X is the mean latency Little's law predicts from the measured throughput.

VUs	Replicas	Requests	Req/s	Mean	N/X	Median	p90	p95	p99	Max
							(ms)			
1	1	1,417	47.23	20.8	21.2	22.1	29.4	31.8	35.8	51.3
	2	1,473	49.10	20.0	20.4	21.4	28.6	30.4	33.0	44.1
10	1	7,382	246.07	40.1	40.6	42.6	57.8	61.8	69.2	95.6
	2	7,365	245.50	40.2	40.7	42.6	58.3	62.5	70.3	100.4
50	1	7,393	246.43	202.3	202.9	211.9	315.9	345.0	400.6	506.3
	2	7,262	242.07	206.0	206.6	214.5	323.7	355.6	416.8	700.9
100	1	7,490	249.67	399.7	400.5	419.3	624.4	673.7	769.7	931.0
	2	7,431	247.70	403.0	403.7	424.5	625.3	680.4	767.6	971.6

Saturation happens at about six clients. One client obtains 47.2 req/s; ten obtain 246 req/s, only 5.2× more; 50 and 100 clients obtain no more at all (246–250 req/s). The bounds of Figure 5a locate the transition without fitting anything. A lone query takes $D = 21.2$ ms, so with no queueing N clients could push $N/D = 47.2\,N$ req/s. One query consumes 45.2 core-milliseconds of server CPU at saturation, so the 12 logical CPUs cap throughput at $X_{\max} = 265$ req/s. The two bounds meet at $N^* = X_{\max}D \approx 5.6$ clients: below that concurrency a client rarely waits, above it every additional client only lengthens the queue. The measured plateau reaches 93–94% of $X_{\max}$, and the remaining CPU went to the load generator, the Docker daemon and WSL2, which were not sampled. The knee lies close to the six physical cores of the host, for the reason explained in Section 7.4.

Latency past the knee is pure queueing. Above N^*, the mean grows linearly with the number of clients, from 40 ms at 10 clients to 202 ms at 50 and 400 ms at 100 (Figure 5b). Two checks show that the system

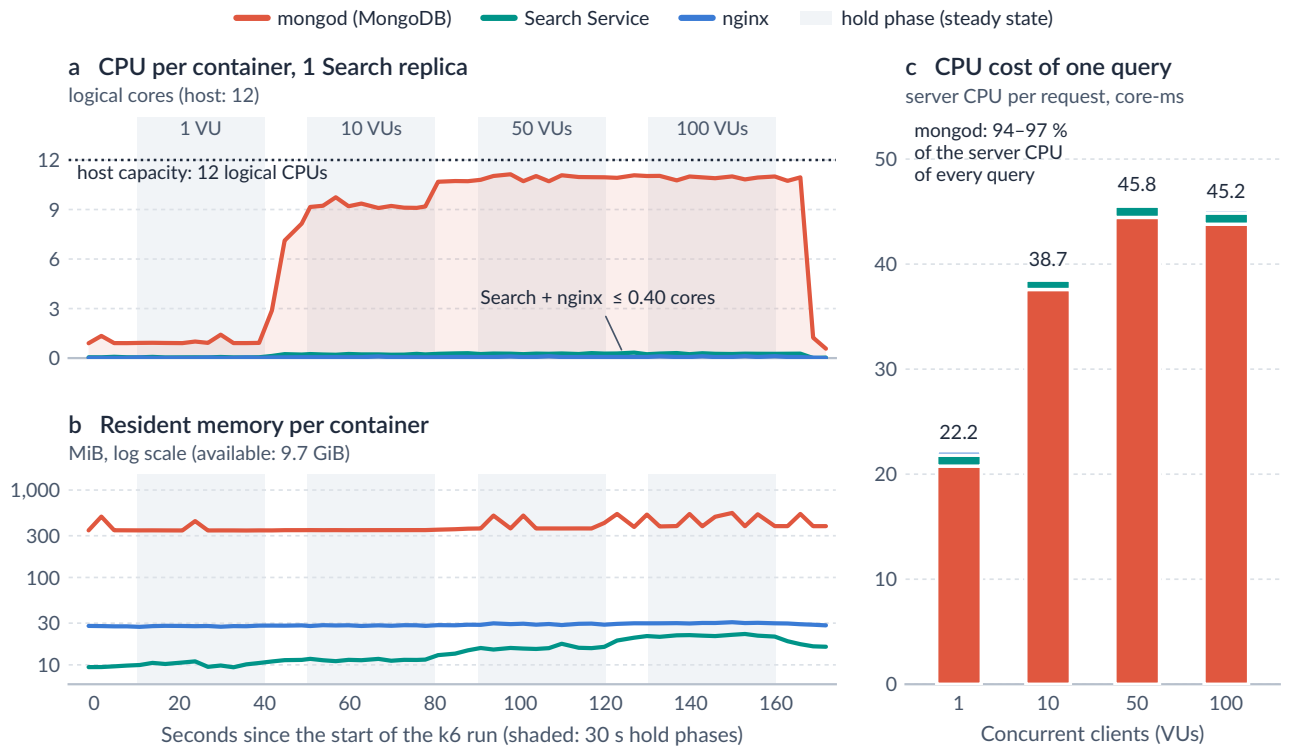


Figure 6: Resources during the one-replica search benchmark (`docker stats`, one sample about every 3 s; shaded: the 30 s hold phases whose means enter Table 17). (a) CPU per container: `mongod` rises from one core to 10.9 of the 12 logical CPUs (dotted) while the Search Service and nginx together never exceed 0.40 cores. (b) Resident memory on a logarithmic scale: every process stays two orders of magnitude below the 9.7 GiB available. (c) Server CPU per query, CPU of each container divided by the throughput of the level (utilisation law): `mongod` accounts for 94–97% of it, and the cost doubles from one client to saturation. The two-replica run is indistinguishable (Table 17).

behaves as one clean bottleneck. First, the means agree with Little's law to within 2% at every level (Table 16), so no work is lost or hidden. Second, they stay 6–7% above the asymptotic lower bound $N/X_{\max}$, and that gap is constant: throughput does not fall as the queue grows, so there is no thrashing, lock convoy or retry storm, only waiting. The tail widens with the queue: the 95th percentile is 1.44 to 1.63 times the median and the 99th 1.62 to 1.89 times, because a query that lands behind a long aggregation waits for it.

Service objectives. The script's objectives are a mean below 200 ms over the whole run and a 95th percentile below 500 ms at every level. The first holds (194.0 and 195.8 ms over the two runs). The second holds up to 50 clients (345 and 356 ms) and fails at 100 (674 and 680 ms), which is why k6 exits with code 99; every availability threshold (fewer than 1% failed requests, more than 99% passed checks) passed. With zero think time the benchmark is deliberately harsher than real traffic. Little's law with a think time Z between a user's searches gives the population a saturated service supports: $N \approx X\,(R + Z) \approx 1{,}250$ users who search every 5 s. This is an estimate derived from the law, not a measurement.

7.4 CPU and memory utilisation per process and host

Figure 6 follows CPU and memory of every container through the one-replica search run, and divides the CPU by the throughput to obtain the cost of one query in each process. Table 17 gives the per-level means for both replica configurations.

The datamart consumes the host. With one client, `mongod` already uses one full core, and its 20.8 ms of CPU per query coincide with the 20.8 ms mean response: one aggregation runs on one core for practically the whole request. From 10 clients onwards `mongod` uses 9.2 to 11.0 cores, and from 50 clients all containers together use 94% of the host (Figure 6a); the remainder is shared by the load generator, the Docker daemon and WSL2, so the host is effectively CPU-saturated. The Search process uses 0.05 to 0.27 cores, about 1 ms of CPU per query, and nginx 0.2 to 0.4 ms. `mongod` accounts for 93.5% of the server-side CPU of a query with one client and 97% from ten clients onwards (Figure 6c). The Search Service is a thin translation layer; the search engine, in terms of where the cycles go, is the aggregation inside MongoDB.

Table 17: CPU (logical cores) and resident memory (MiB, mean and maximum) per container during the hold phases of the search benchmark. The host provides 12 logical cores; with two replicas, the Search columns give replica 1 + replica 2. Values are those written by the chart script to `chart_data.csv`.

		CPU (logical cores)				Memory (MiB)		
Replicas	VUs	`mongod`	Search	nginx	Total (host)	`mongod`	Search	nginx
1	1	0.98	0.05	0.02	1.05 (9%)	356 (442)	10	28
	10	9.25	0.22	0.06	9.53 (79%)	348 (348)	11	28
	50	10.95	0.27	0.07	11.29 (94%)	403 (511)	16	29
	100	10.94	0.27	0.08	11.29 (94%)	448 (547)	22	30
2	1	0.99	0.03 + 0.03	0.02	1.06 (9%)	387 (388)	14 + 9	28
	10	9.23	0.13 + 0.13	0.06	9.55 (80%)	390 (391)	14 + 11	28
	50	10.85	0.14 + 0.15	0.08	11.23 (94%)	441 (545)	16 + 13	29
	100	10.93	0.14 + 0.14	0.07	11.28 (94%)	474 (579)	18 + 17	30

Why the cost of a query doubles under load. The CPU cost of one query is not constant: it rises from 20.8 core-ms with one client to 37.6 with ten and 44.4 with fifty (Figure 6c), a factor of 2.1. Between 10 and 50 clients `mongod` received 1.7 more cores but served no more queries. The factor of about two is the signature of simultaneous multithreading: the host has six physical cores presented as 12 logical CPUs, and once more aggregations are runnable than there are physical cores, two hardware threads share the execution units of one core, each progressing at roughly half speed while `docker stats` charges both as busy. The consistent arithmetic is that six physical cores running one aggregation each at the single-client cost would deliver 6/20.8 ms ≈ 290 req/s; the measured plateau is 86% of that, a gap compatible with a lower all-core clock than the single-core turbo and with the CPU taken by k6. Clock frequency and hardware counters were not recorded, so this mechanism is a well-supported hypothesis rather than a measurement. Its practical reading matters more than its proof: on this host, *logical* CPUs are not capacity, and the effective processor count for aggregation-heavy work is closer to six than to twelve.

Why one query costs so much. The datamart micro-benchmarks (Table 18) isolate the aggregation on a synthetic 300-book datamart. Fetching the posting list of a term contained in every book takes 0.60 ms, but the full search aggregation for that term takes 26.2 ms, 43× more. The cost lies in the stages after the intersection: one `$lookup` sub-pipeline per candidate book, followed by a `$facet` that counts and sorts every match. A common term has a candidate for almost every book, so the join runs about 300 times per query. Adding the three metadata filters makes the query *slower* (37.9 ms, +45%), because the filters are evaluated per candidate inside the lookup, after the point read by `_id`, instead of pruning candidates through the filter indexes. This reading of the query plan is an inference from the pipeline's structure; `explain()` output was not recorded.

Table 18: Datamart micro-benchmarks against the live server (`go test -bench`, 10 samples, median ± 95% confidence interval from `benchstat`; `GOMAXPROCS` = 12). The read benchmarks use a synthetic 300-book datamart in which the queried term occurs in every book; bytes and allocations are those of the Go client.

Benchmark	Operation	Time per op	Bytes/op	Allocs/op
`PostingsForTerms`	Posting list of one term	603.0 μs ± 5%	54.4 KiB	1,973
`SearchBooksWithoutFilters`	Full search aggregation, no filter	26.18 ms ± 3%	34.1 KiB	773
`SearchBooksWithAllFilters`	Same, with author, language and year	37.90 ms ± 12%	35.6 KiB	679
`IndexBook`	Re-index a synthetic 7-token book	4.544 ms ± 4%	42.4 KiB	582
`IndexBookRealBook`	Re-index book 1342 (6,576 terms)	636.9 ms ± 2%	42.4 MiB	444,400

Memory is not the constraint, yet. Figure 6b uses a logarithmic axis because the three processes differ by two orders of magnitude. `mongod` held 348–474 MiB on average with transient peaks up to 547 MiB (579 MiB with two replicas) during the saturated holds, consistent with per-query sort and `$facet` buffers being allocated and released; the Search process grew from 10 to 22 MiB with concurrency, as each in-flight request holds a goroutine stack and a response buffer; nginx stayed at 28–31 MiB; the Indexing Service stayed below 120 MiB (Table 15). The whole stack used less than 0.7 GiB of the 9.7 GiB available. The thread counts reflect the two concurrency models of Section 3.3: `mongod` grew from about 45 to 155 threads (209 in the two-replica run) as connections were opened, one thread per connection, while each Search process stayed at 13 to 24 operating-system threads for up to 100 concurrent requests. A 300-book datamart fits entirely in MongoDB's cache; memory becomes relevant when the working set of posting arrays outgrows it, which this corpus does not

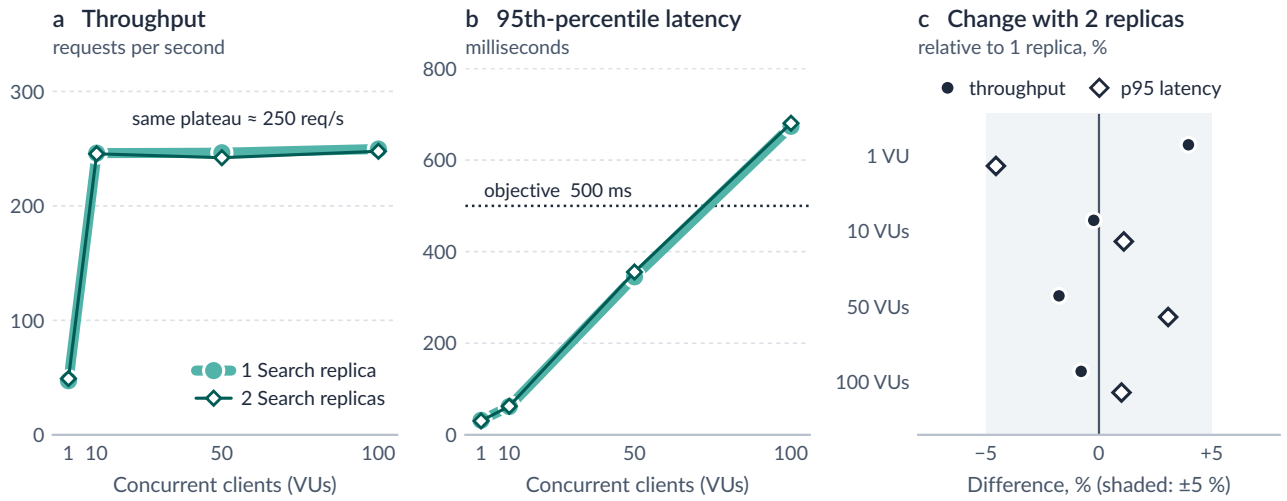


Figure 7: One versus two Search replicas behind nginx (`least_conn`), identical k6 profile, seed, datamart and host. (a) Throughput and (b) 95th percentile latency: the thin dark line of the two-replica run lies inside the thick light line of the one-replica run at every level. (c) Relative change with two replicas; the shaded ±5% band is a visual reference, not a measured confidence interval. No change has a consistent sign.

exercise.

7.5 Effects of replication

The replica experiment (item RF 8 of the group's Stage 2 audit checklist, the label used in `docker-compose.replicas.yml` and `deploy/nginx/nginx.conf`) repeated the identical k6 profile, with the same seed, datamart, host and proxy, once with one and once with two Search replicas behind nginx; the replica count was the only variable (Figure 7).

The load was distributed. nginx split the work evenly: from 10 clients upwards the two replicas used the same CPU within 7% (0.132/0.127, 0.144/0.153 and 0.137/0.138 cores at 10, 50 and 100 clients), and the `X-Upstream` header alternated between the two container addresses. Their combined CPU (0.26–0.30 cores) is what one replica used alone (0.22–0.27 cores): the work was divided between two processes, not reduced.

Throughput and latency did not change. Throughput changed by +4.0%, −0.2%, −1.8% and −0.8% at 1, 10, 50 and 100 clients, and the 95th percentile by −4.6%, +1.1%, +3.1% and +1.0% (Figure 7c). Every difference lies within ±5%, the sign flips between levels, and the one-client run, where two replicas look best, is the level at which no replica is busy. With one run per configuration these differences are indistinguishable from run-to-run variation, so no effect of replication is detectable; `mongod`'s CPU changed by less than 1% at every level.

Why it could not change. The asymptotic bound explains the null result quantitatively. $X_{\max} = m/D_{\text{cpu}}$ depends on the processors of the host and on the CPU one query consumes; a second Search replica changes neither, because it only splits the Search share of D_{cpu}, 1.1 of 45.2 core-ms, between two processes. Even if the Search and nginx tiers cost nothing at all, the bound would rise from 265 to 274 req/s, +3%: that is the most any amount of Search replication can buy on this host. Replication did change one property of the datamart: each replica brings its own connection pool, so `mongod` ran up to 209 threads instead of 155, with no more CPU to run them. The measurement thus confirms the expectation of Section 6.7.

7.6 Effects of threading

Figure 8 contrasts three kinds of work under added threads or workers: CPU-bound text processing, network-bound ingestion and datamart-bound operations.

CPU-bound work scales, up to the physical cores. The three text functions run in parallel without shared state and reach 3.4× to 4.6× with eight threads (Table 19, Figure 8a). Scaling is close to linear up to two threads and flattens beyond four, as expected on six physical cores with simultaneous multithreading, the same

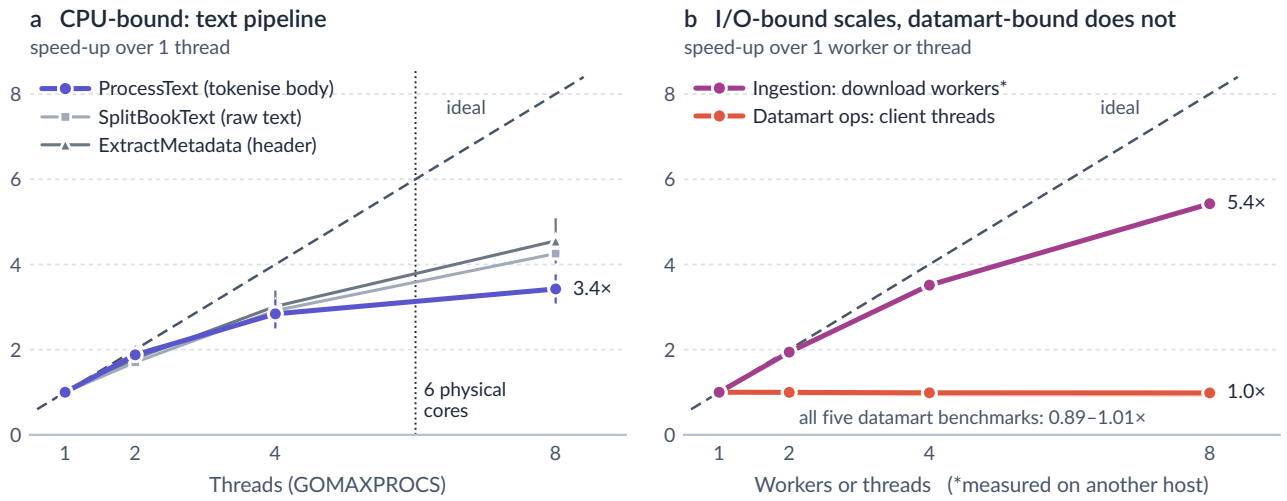


Figure 8: Threading. (a) The three CPU-bound functions of the indexing pipeline run in parallel (`b.RunParallel`) over the 383 real books; error bars are the 95% confidence intervals of `benchstat` combined for the ratio. They scale to 3.4–4.6× with eight threads, flattening beyond the six physical cores. (b) Network-bound ingestion of 99 real books into a cold datalake scales to 5.4× with eight download workers (measured on another machine, Table 13), whereas the five datamart micro-benchmarks stay at 0.89–1.01× whatever the client's thread count: their time is spent inside `mongod`. Dashed: ideal linear speed-up.

Table 19: Text pipeline micro-benchmarks over the 383 real books of the datalake, run in parallel by `GOMAXPROCS` goroutines (`b.RunParallel`): median time per operation (10 samples) and speed-up.

Function	Input	1 thread	2	4	8	Speed-up at 8	Efficiency
`ProcessText`	book body	13.66 ms	7.27 ms	4.81 ms	3.99 ms	3.42×	43%
`ExtractMetadata`	header	23.5 μs	13.1 μs	7.79 μs	5.16 μs	4.55×	57%
`SplitBookText`	raw text	211 μs	124 μs	72.5 μs	49.7 μs	4.25×	53%

effect that doubles the cost of a query in Section 7.4. `ProcessText`, the tokeniser, scales least: it allocates about 15 MB per book, so memory bandwidth and garbage collection are plausible further limits. Text processing is in any case not what limits indexing (Section 7.2).

Datamart-bound work does not scale with client threads. The datamart micro-benchmarks were swept with `-cpu 1,2,4,8`; their speed-ups stay between 0.89 and 1.01 (Figure 8b), and, for example, `SearchBooksWithoutFilters` takes 26.22–26.32 ms at every setting. These benchmarks issue one request at a time, so the sweep varies the client's thread count, not the load on the server. The flat result shows that the cost of a datamart operation is spent in `mongod`, not in the Go client; how the server scales with concurrent requests is what the k6 runs measure.

Network-bound work scales almost linearly. Ingestion scales almost linearly up to four download workers (3.51×) and reaches 5.42× (21.3 books/s) at eight, with the Ingestion process using on average less than 2% of the machine's CPU (Table 20). Its limit is the latency of the upstream Gutenberg server, not local resources. Because the experiment ran on another machine, the shape of the curve is the finding, not its absolute rates.

Table 20: Cold-datalake ingestion of 100 real Gutenberg IDs (99 available, one answered `404 book_unavailable`); only `2xx` answers are counted. Host: Intel Core Ultra 7 155H (16 cores), 32 GB, Windows 11.

Workers	Elapsed (s)	Books/s	Speed-up	CPU (machine)	Peak working set
1	25.19	3.93	1.00	0.5%	83 MB
2	12.98	7.63	1.94	0.6%	37 MB
4	7.17	13.81	3.51	1.1%	37 MB
8	4.64	21.32	5.42	1.8%	47 MB

Configurations not tested. Some combinations remain unmeasured, and the conclusions do not extend to them. Ingestion and Indexing replicas were not run under load (Indexing ran as two replicas only in the lease

experiment). Every search run went through nginx, so the proxy's own latency was not isolated. The services' `GOMAXPROCS`, the driver's pool size and the eight-slot bounds were not varied, and no container ran with CPU or memory limits or pinned to cores.

7.7 Coordination overhead

The Control Module logs the wall-clock duration of every ingestion and indexing step it waits for. The time of a run that is *not* spent waiting on a service step, its worker time minus the step totals, is therefore an upper bound on the orchestrator's own overhead: claiming, recording progress under `flock`, logging and retry backoff. Table 21 reports four runs of 100 books with 1, 2, 4 and 8 workers. These runs predate the official session and used different ID ranges, so they are reported as a table of indicative values rather than plotted next to the official curves.

Table 21: End-to-end orchestration runs (29 October 2026). Step means are over the books that entered the pipeline; "unattributed" is the share of worker time not spent waiting on a service step.

Workers	IDs	Indexed / failed	Elapsed (s)	Books/s	Ingest step	Index step	Unattributed
1	1–100	96 / 4	219.2	0.44	0.59 s	1.41 s	8.8%
2	101–200	78 / 22	187.2	0.42	0.45 s	2.37 s	24.9%
4	201–300	98 / 2	105.8	0.93	0.44 s	3.64 s	3.6%
8	301–400	100 / 0	90.9	1.10	0.43 s	6.65 s	2.6%

The orchestrator itself is cheap: in the run without failures, at most 2.6% of worker time was spent outside service calls, and that bound also includes the idle tail at the end of the run. The higher shares at one and two workers come from retry backoff: those runs predate the realignment of the ingestion entry point with its specification, when books that Gutenberg does not serve were answered with `503` and retried as transient before being recorded as failed (22 such books in the two-worker run). The real coordination cost is the waiting. Going from one to eight workers raised end-to-end throughput only 2.5× (0.44 to 1.10 books/s), because the mean index step grew 4.7× (1.41 to 6.65 s), the same datamart contention that Figure 4b measures as a rising per-book cost, while the mean ingestion step stayed between 0.43 and 0.59 s. In other words, adding workers to the coordinator moves the queue from the coordinator into MongoDB; it does not shorten it.

7.8 Bottlenecks and performance limits of the single host

The figures rank the limits of the deployment in the order in which they bind:

1. **Search: datamart CPU.** About 250 req/s from ten clients on, with the knee at $N^* \approx 5.6$ clients (Figures 5 and 6).
2. **Host CPU, counted in physical cores.** Six physical cores, not twelve logical CPUs, are the real capacity (Section 7.4).
3. **Indexing: write contention in the datamart, not capacity.** 48% efficiency at ten updates with the host 68% busy (Figure 4).
4. **Ingestion: the upstream network.** Downloads parallelise well at negligible local cost (Figure 8b).
5. **Memory: not yet a limit.** Under 0.7 of the 9.7 GiB available for a 300-book datamart (Figure 6b).
6. **Coordination: not a throughput limit.** At most 2.6% of worker time (Table 21).
7. **Co-location.** One CPU shared by load generator, datamart and services, so the ceiling belongs to the whole host.

7.9 Threats to validity

- **One run per configuration.** The k6 results have no run-to-run confidence intervals; differences of a few percent, such as those between one and two replicas, cannot be distinguished from noise.
- **Short holds.** The 30 s hold phases capture steady state but not long-run effects such as cache eviction or the growth of the WiredTiger cache.
- **Co-located load generator.** k6 competed with the services for CPU, especially at 100 VUs, and its own consumption was not sampled.
- **Sampling.** `docker stats` yields one sample about every 3 s, each covering a short interval, so the per-level CPU values are means of 8 to 10 samples. During the saturated holds the coefficient of variation of

`mongod`'s CPU stayed below 2.5%; the small Search and nginx readings vary more (up to about 30%), which is why their per-query costs are quoted to one significant digit.

- **Virtualisation and power management.** WSL2 adds a hypervisor layer, and CPU frequency was neither fixed nor recorded; the simultaneous multithreading explanation of Section 7.4 rests on that missing measurement.
- **Warm caches.** The page cache and MongoDB's cache were warm, and the indexing benchmark re-indexed books already present.
- **Corpus.** The search datamart was seeded by walking Gutenberg IDs upwards from 1 until 300 books were published; results may shift on a corpus with a different term distribution or size.
- **Unrecorded internals.** MongoDB's `serverStatus` counters and `explain()` plans were not captured; attributions to write conflicts and to the lookup stage are inferences.
- **Heterogeneous evidence.** The ingestion experiment ran on another machine, the coordination runs before the official session, and the API validation against MongoDB 8.3.7; these results are never compared numerically with the official ones.

8 Conclusions and Future Improvements

Stage 2 met its objectives: the Stage 1 data layer now runs as cooperating services with documented and validated contracts, an explicit coordination mechanism and a measured performance envelope. The measurements also close the question Stage 1 left open: MongoDB serves several independent processes correctly under concurrent access, and on a single host it is the first component to saturate on both the read and the write path (Section 7.8), the fact from which the design of Stage 3 follows.

8.1 Strengths

- **Contracts that are specified and verified.** The OpenAPI documents are generated from the code, and the running stack passes every API check, automated test and end-to-end script (Sections 4.8 and 5.7).
- **Coordination that tolerates failures.** Ingestion before indexing is enforced twice, half-indexed books are invisible, rebuilds are serialised across replicas, and idempotency turns a crash between a call and its record into a harmless replay (Sections 3.2 and 5.6).
- **Graceful saturation.** Past the knee, throughput holds its plateau and latency grows exactly as queueing predicts, with no failed request even at 100 clients (Section 7.3).
- **A lean service tier.** The Go services and nginx account for only a few percent of the CPU of a query; the cost lies in the datamart (Section 7.4).
- **Concurrency that is bounded and measured.** Semaphores, timeouts and context propagation keep overload from exhausting resources, and CPU- and network-bound work scale with threads as expected (Section 7.6).
- **Reproducible evidence.** Every figure is regenerated by one script from committed raw files (Section A).

8.2 Weaknesses

- **One datamart server bounds both paths.** Search saturates early and misses its 95th-percentile objective at 100 clients, indexing scales sub-linearly, and Search replicas cannot help on one host (Section 7.8).
- **An expensive query plan.** The per-candidate `$lookup` and the `$facet` dominate the cost of a search, and metadata filters make it slower rather than faster (Section 7.4).
- **Write amplification and contention.** The one-document-per-term layout rewrites thousands of posting arrays per book, concurrent updates collide on shared term documents, and common terms approach the document size ceiling (Sections 6.4 and 7.2).
- **Per-process coordination state.** The rebuild replay list, the ingestion single-flight map and the Control Module's claim cursor live in memory, so they hold for one process only (Table 23).
- **One coordinator.** New books stop entering the pipeline while the Control Module is down (Section 6.1).
- **Limited evidence.** Single runs, a co-located load generator and unrecorded server counters limit the precision of the results (Section 7.9).

8.3 From the measurements to the Stage 3 evolution

The figures cannot show how the system will scale on a cluster (Section 2), but they rule options out and set priorities. Table 22 orders the Stage 3 work by expected impact and pairs each decision with the measured fact behind it and with the acceptance check that will confirm it, using the same benchmark and the same figure, so that Stage 3 is evaluated against Stage 2 like for like.

Table 22: Prioritised improvements for Stage 3: the measured facts behind each decision, and how each decision will be verified with the same benchmarks and figures.

#	Measured fact	What it implies	Stage 3 decision	Acceptance check
1	`mongod`: 94–97% of the CPU of a query; ceiling 250 req/s at $N^* \approx 6$ clients; full aggregation 43× the cost of fetching postings (Figures 5 and 6 and Table 18)	The read ceiling is a property of the datamart, and a cheaper query raises it on every node	Rank the intersected postings and join only the requested page; filter on denormalised or index-backed attributes; cache hot queries in the Search replicas, keyed by index version; then scale reads with replica-set secondaries or shards	Figure 6c: lower core-ms per query, checked with `explain()`; Figure 5a: plateau scales with data nodes
2	Indexing: 48% efficiency with the host 68% busy; per-book `mongod` cost +58% (Figure 4)	Contention on shared term documents, not lack of CPU: more cores on one server would not help	(term, book) posting documents (`docs/mongodb.md`), which also remove the 16 MB ceiling; shard key hashed on the term; batched, unordered writes; indexing tasks partitioned by book	Figure 4b: per-book cost flat ($\leq$ 10% growth) from 1 to 10 clients; efficiency $\geq$ 80% at 10
3	Query cost doubles past six clients on 6 physical / 12 logical cores; single runs; server counters and CPU frequency not recorded (Figure 6c, Section 7.9)	Logical CPUs overstate capacity, and two attributions remain hypotheses	Size data nodes by physical cores; pin or limit containers; run k6 on its own node; record `serverStatus`, query plans and CPU frequency; repeat every configuration at least three times	Cost per query constant across load levels; every figure with confidence intervals
4	Orchestrator overhead $\leq$ 2.6%; index step 4.7× longer with 8 workers (Table 21)	Central coordination is an availability problem, not a throughput one	A tasks collection with leases replacing the control files and the in-memory claim, or a broker with consumer groups	Throughput unchanged when one coordinator is killed mid-run
5	Deep offsets cost datamart work proportional to the offset (Section 6.3)	Offset pagination degrades as the corpus grows	Keyset pagination, continuing after the last returned score and book ID	Page latency independent of the offset
6	Two Search replicas: $\lvert\Delta\rvert \leq 5\%$; at most +3% possible on one host (Figure 7)	Stateless replicas add availability, not throughput, while the datamart is shared	Keep ≥ 2 Search replicas on different nodes for availability; add more only when Figure 6c shows the Search tier is the bottleneck	Repeat Figure 7 on the cluster: throughput should rise with replicas once data nodes scale
7	Ingestion 5.4× with 8 workers at $<$ 2% CPU (Figure 8b)	Network-bound; scales with concurrency, not with machines	Keep one Ingestion replica per node with a worker pool; rate-limit towards Gutenberg	Books/s versus workers per node

Why a cheaper query comes first. Because the read bound is $X_{\max} = m/D_{\text{cpu}}$, read capacity can grow through more processors m (data nodes that serve reads) or a smaller demand D_{cpu} per query, and the second multiplies the first: every halving of the 21 ms per query doubles the ceiling of every data node. As a rough illustration, sustaining 1,000 req/s on today's plan would need at least four six-core data nodes serving reads, an extrapolation of the bound rather than a measurement. Once the datamart scales, the Search tier, at about 1 ms of CPU per query or roughly 1,000 queries per second per core, becomes the next limit, and only then do Search replicas on separate nodes raise throughput.

Cross-machine communication and storage access. The HTTP/JSON contracts carry over unchanged, because every address already comes from configuration, and each local mechanism is replaced as listed in Table 23: for example Kubernetes services or Consul for discovery, and MinIO or S3 for the datalake, with a completion marker object written last. Timeouts and retry budgets must be recalibrated from the measured percentiles rather than single-host defaults; the 21 ms service time of a lone query leaves room for a few milliseconds of network per hop. Idempotency keeps retries over an unreliable network safe, and jitter in the backoff prevents synchronised retry storms. MongoDB becomes a replica set of at least three nodes for availability and read scaling, with the inverted index laid out and sharded as in row 2 of Table 22.

Distribution of tasks and requests. Pipeline work becomes claimable tasks, either documents claimed atomically under a lease or `BookDownloaded` and `BookIndexed` events partitioned by book ID, so that the updates of one book stay ordered and several Control instances (or none, under choreography) can share the work without duplicating it. Search requests are balanced over replicas on several nodes, and read-heavy traffic can be served by replica-set secondaries where bounded staleness is acceptable; Search replication then pays off, because the datamart scales with it.

Consistency. The per-book publication protocol (pending, writes, published) remains the unit of visibility. On a replica set the publication steps use `w: majority`, so that a published book cannot be rolled back after a failover, search reads use `readConcern: majority`, and the `-verify` mode uses causally consistent sessions to read its own writes. The version pointer that replaces `renameCollection` is resolved once per request, which also closes the sub-millisecond mixed window of the current swap.

Recovery from unavailable nodes. Health checks split into liveness and readiness, readiness including MongoDB connectivity, so that load balancers stop routing to a replica that cannot serve, as nginx already does by ejecting a replica after three failures. Driver-level retryable reads and writes absorb replica-set elections, expired leases let another node take over the work of a crashed holder, coordinators resume from durable task state, and idempotency makes redelivery harmless. Fencing tokens on writes made under a lease stop a paused former holder from overwriting the work of its successor.

Table 23 lists every Stage 2 assumption that holds only on one machine, including the in-process locks and the shared files used for coordination, together with its Stage 3 replacement.

Table 23: Single-host assumptions and their Stage 3 replacements.

Stage 2 assumption	Where	Why it breaks on a cluster	Stage 3 replacement
Datalake on a shared named volume	Ingestion, Indexing	A volume exists on one node	Object storage or distributed file system
Completeness by atomic local rename	datalake	Rename semantics are local to one file system	Conditional writes; marker object last
`flock` on control-state files	Control Module	File locks do not cross machines	Records in the database with unique keys
In-memory claim; one Control instance by convention	Control Module	Two coordinators would duplicate work	Atomic claims with leases, or broker consumer groups
In-process single-flight map	Ingestion	Deduplicates within one replica only	Per-book task claim (duplicates remain harmless)
In-process rebuild flag and replay list	Indexing	Updates sent to another replica during a rebuild are not replayed	Durable pending-updates collection or version-tagged writes
Per-process semaphores (8 slots)	Ingestion, Indexing	Admission is per replica, not global	Global rate limiting through queue depth
Lease expiry from client clocks	rebuild lease	Clock skew between nodes	Server time and fencing tokens
Compose DNS names and `localhost` ports	deployment	One Docker network on one host	Service discovery and a load balancer per tier
Two `renameCollection` commands	rebuild publication	Unsupported on sharded collections	Version pointer document
One `mongod`	datamart	Single point of failure and CPU ceiling	Replica set and sharding
Load generator on the service host	benchmarks	Skews every measurement	Dedicated load-generation nodes

A Evidence Provenance

Every number and figure of Section 7 can be regenerated from the repository root: `docs/evidence/benchmark_protocol.md` holds the exact commands of the official session and `docs/evidence/README.md` the per-result steps. Every figure is drawn by `scripts/generate_charts.py` (pandas, matplotlib, seaborn) directly from the raw files of Table 24, and the script also writes `docs/figures/chart_data.csv`, which lists every plotted value and every derived quantity quoted in the text. The k6 outputs in `test/load/results/` are byte-identical to the copies in `docs/evidence/` that the script reads.

Table 24: Provenance of the results (paths relative to the repository root). Every figure is `docs/figures/<name>.pdf`.

Result	Raw evidence
Table 12	`docs/evidence/README.md` (environment), `benchmark_protocol.md` (host record)
Tables 14 and 15, Figure 4 (`fig2_indexing_throughput`)	`docs/evidence/indexing_benchmark_2026-11-01T16-14-15-954Z.{csv,json}`, `indexing_benchmark_run.log`, `indexing_resources.csv`
Table 16, Figure 5 (`fig1_query_latency`), Figure 7 (`fig4_replication`)	`docs/evidence/search_benchmark_2026-11-01T16-26-57-365Z_1replica.{csv,json}`, `search_benchmark_2026-11-01T16-30-41-872Z_2replicas.{csv,json}`, `search_benchmark_run_{1replica,2replicas}.log`
Table 17, Figure 6 (`fig3_resource_utilisation`), bound $X_{\max}$ of Figure 5	`docs/evidence/search_resources.csv`, `search_resources_2replicas.csv`, aligned with `setup_data.startedAt` of each k6 JSON
Tables 18 and 19, Figure 8 (`fig5_threading`)	`docs/evidence/benchstat{,_cpu,_micro}.csv`, `benchstat*.txt`, `benchmarks*.txt`, `microbenchmarks.txt`
Table 20, Figure 8b	`docs/evidence/ingestion_throughput.md`, `ingestion_throughput_summary.csv`, `ingestion_resources_w*.csv`
Table 21	`cmd/control/results/coordination_workers_{1,2,4,8}.txt`
Rebuild figures	`docs/evidence/rebuild_replica.md`
Table 7 and tests	`docs/evidence/api_validation_results.md`, `e2e_control_*.txt`, `e2e_compose_*.txt`, `tests.txt`